AI CERTs®

AI Certification Program

AI+ Cloud Practitioner™

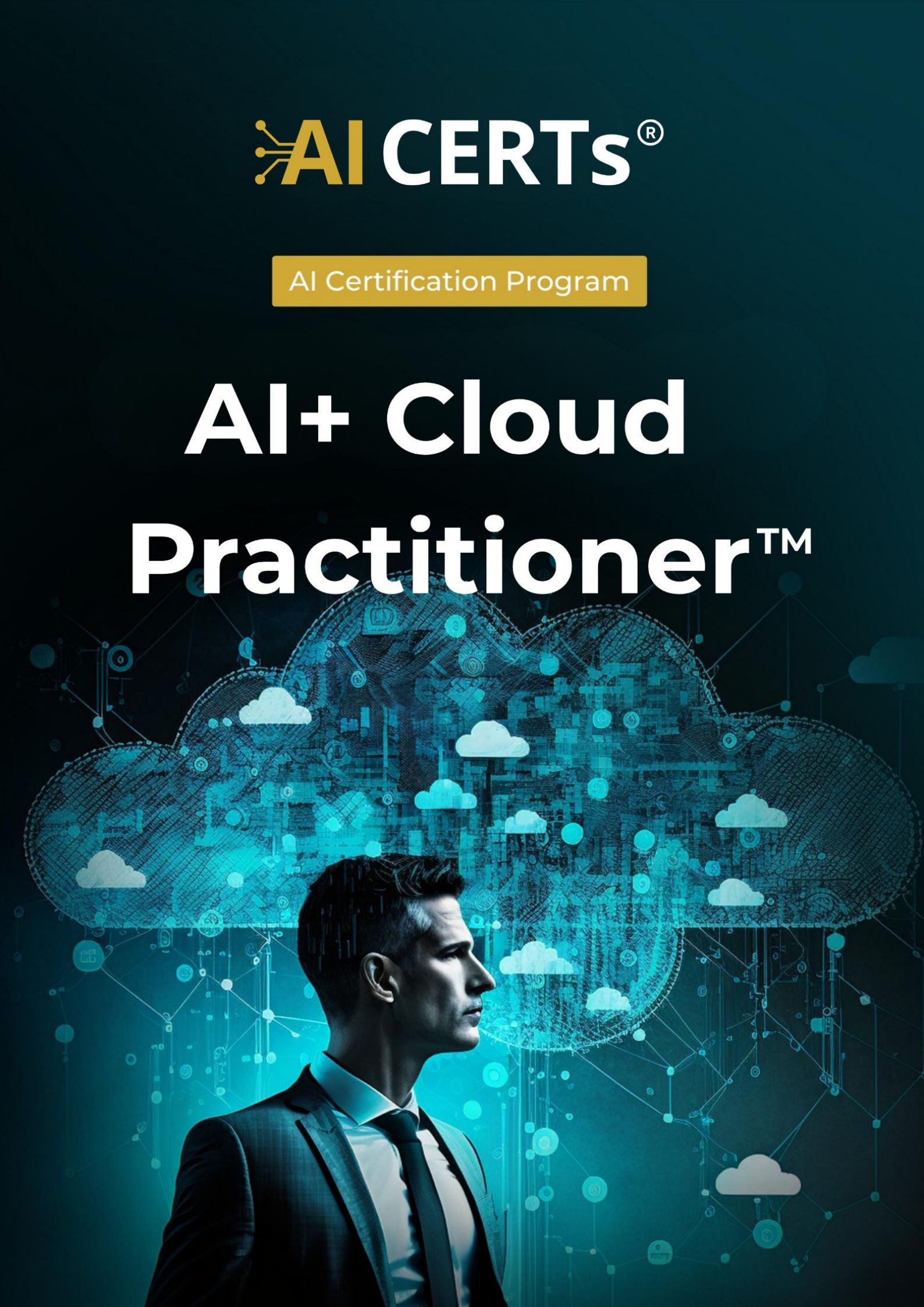


Table of Contents

Table of Contents 2

Retrieval-Augmented Generation (RAG) 3

8.1 RAG architecture and components 4

8.2 Vector databases and embeddings 13

8.3 Advanced RAG patterns 36

8.4 Evaluation metrics for RAG systems 51

8.5 Cloud-native vector search services 59

8.6 Use Cases 69

8.7 Case Studies 76

Copyright © 2026. All rights reserved

Module 8

Retrieval-Augmented Generation (RAG)

Overview

Retrieval-Augmented Generation (RAG) is a powerful AI framework that enhances the capabilities of large language models (LLMs) by combining them with external knowledge retrieval mechanisms. Instead of relying solely on pre-trained knowledge, RAG retrieves relevant information from structured and unstructured data sources at query time and incorporates it into the generation process, enabling more accurate, up-to-date, and context-aware responses. This approach addresses common limitations of LLMs, such as hallucinations, outdated knowledge, and lack of domain-specific expertise, making RAG an essential technique for building reliable enterprise AI applications, intelligent assistants, and knowledge management systems.

This module covers:

- **RAG Architecture and Components:** Understand the end-to-end workflow and key building blocks of Retrieval-Augmented Generation systems.
- **Vector Databases and Embeddings:** Learn embedding concepts and explore vector databases such as **Pinecone**, **FAISS**, and **Weaviate** for efficient semantic search.
- **Advanced RAG Patterns:** Study advanced techniques including **Advanced Chunking**, **HyDE**, and **GraphRAG** to improve retrieval quality and contextual reasoning.
- **Evaluation Metrics for RAG Systems:** Evaluate RAG performance using **RAGAS**, **faithfulness**, **relevance**, and other quality assessment metrics.

Copyright © 2026. All rights reserved

- **Cloud-Native Vector Search Services:** Explore managed cloud services for scalable vector storage, indexing, and retrieval.

- **Use Cases:** Examine practical applications of RAG across enterprise knowledge management, customer support, search, and AI assistants.

- **Case Studies:** Analyze real-world implementations of RAG systems to understand design choices, deployment strategies, and business impact.

8.1 RAG architecture and components

Retrieval-Augmented Generation (RAG) is an AI framework that combines the knowledge retrieval capabilities of information retrieval systems with the reasoning and text generation capabilities of Large Language Models (LLMs). Instead of relying solely on the model's pre-trained knowledge, RAG retrieves relevant information from external knowledge sources at query time and uses it to generate accurate, context-aware, and up-to-date responses. This architecture significantly reduces hallucinations, improves factual accuracy, and enables domain-specific question answering.

Why RAG?

Traditional LLMs have several limitations:

- Knowledge is limited to the data used during model training.

- Models cannot access newly added or frequently changing information.

- Responses may contain hallucinations or incorrect facts.

- Fine-tuning models for every knowledge update is expensive and time-consuming.

Copyright © 2026. All rights reserved

RAG addresses these challenges by retrieving relevant information from external knowledge repositories before generating responses.

RAG Architecture:

RAG Architecture is the structural framework that integrates information retrieval with large language models (LLMs) to generate accurate, context-aware, and knowledge-grounded responses. It defines the end-to-end workflow and key components involved in retrieving relevant information from external knowledge sources and using it to enhance the response generation process.

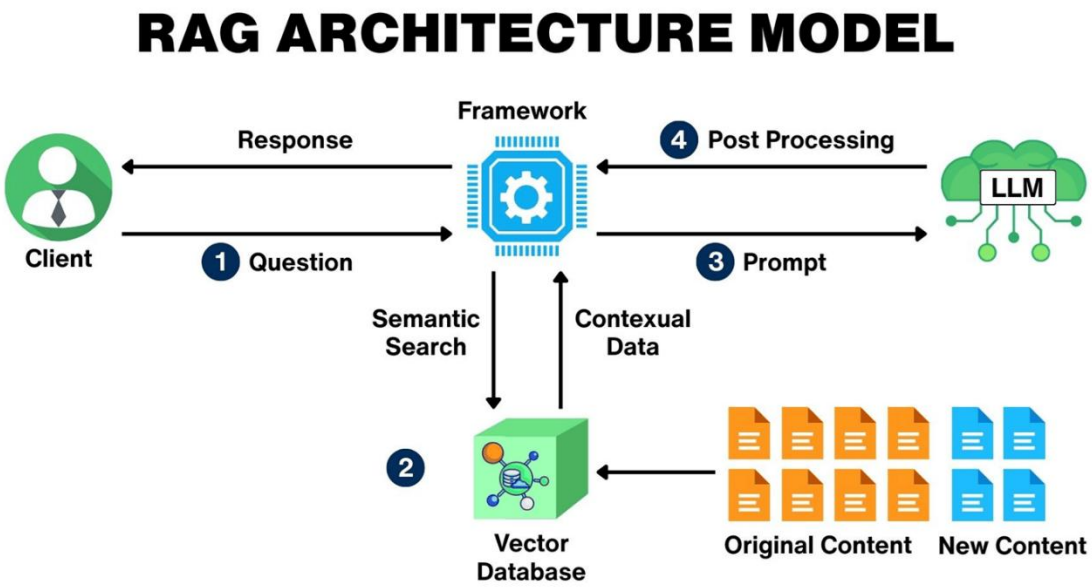


Figure. 8.1: RAG Architecture

The figure illustrates the online workflow of a Retrieval-Augmented Generation (RAG) architecture, where an LLM generates responses by retrieving relevant information from an external knowledge base instead of relying only on its pre-trained knowledge.

1. **Step 1: User Query:** The client submits a question to the RAG framework.

2. **Step 2: Semantic Retrieval**

Copyright © 2026. All rights reserved

- The framework converts the query into an embedding and performs a semantic search on the vector database.

- Relevant information is retrieved from indexed original and newly added documents.

3. **Step 3: Prompt Augmentation**

   - The retrieved context is combined with the user's query to create an augmented prompt.

   - The prompt is sent to the Large Language Model (LLM).

4. **Step 4: Response Generation**

   - The LLM generates a context-aware response using the retrieved information.

   - The framework post-processes the response (e.g., formatting, validation) before returning it to the user.

Core Components of RAG System:

A Retrieval-Augmented Generation (RAG) system consists of several interconnected components that work together to retrieve relevant information from external knowledge sources and generate accurate, context-aware responses. Each component plays a specific role in ensuring efficient retrieval, effective prompt construction, and reliable response generation.

Copyright © 2026. All rights reserved



Figure. 8.2: Key Components of RAG Architecture

The figure illustrates key components of a RAG system which is summarized in below table. The table outlines the key components of a RAG system, their functions, and their roles in enabling efficient retrieval and grounded response generation.

Table 1: Key Components of RAG Architecture

Component	Description	Role in RAG System
Retriever	Searches the knowledge base using semantic search to identify the most relevant information for a user's query.	Retrieves contextually relevant document chunks from the vector database.
Generator	Receives the retrieved context and the user's query to generate responses using a Large Language Model (LLM).	Produces accurate, coherent, and context-aware responses.

Copyright © 2026. All rights reserved

Knowledge Base	Stores structured and unstructured organizational data, such as documents, manuals, FAQs, databases, and web content.	Serves as the primary source of knowledge for information retrieval.
Integration Layer	Coordinates interactions between the retriever, knowledge base, and generator while managing workflows and APIs.	Orchestrates retrieval, prompt construction, response processing, and system integration.

Did you know?
Unlike traditional search engines, RAG retrieves information based on **semantic similarity**, not just keyword matching.

Quick Fact:
RAG reduces hallucinations by grounding responses in retrieved external knowledge.

Benefits of RAG:

Retrieval-Augmented Generation (RAG) offers several advantages over standalone Large Language Models by combining external knowledge retrieval with text generation.

Copyright © 2026. All rights reserved

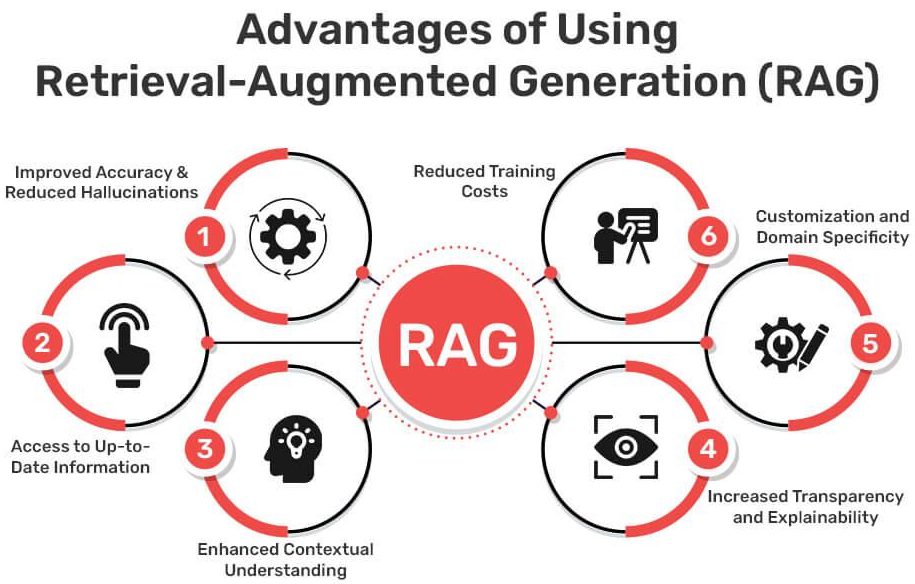


Figure. 8.3: Advantages of RAG Architecture

The figure illustrates key advantages of a RAG system which is summarized in below table.

Table 2: Key Advantages of RAG Architecture

Benefit	Description
Improved Accuracy & Reduced Hallucinations	Grounds responses in retrieved information, improving factual accuracy.
Access to Up-to-Date Information	Retrieves the latest knowledge without retraining the LLM.
Enhanced Contextual Understanding	Uses relevant context to generate more meaningful responses.

Copyright © 2026. All rights reserved

Increased Transparency & Explainability	Supports responses with retrieved sources, improving trust.
Customization & Domain Specificity	Leverages domain-specific knowledge for specialized applications.
Reduced Training Costs	Updates the knowledge base instead of retraining the model, saving time and cost.

Use Case

Title: Building an Enterprise Knowledge Assistant Using Retrieval-Augmented Generation (RAG)

Scenario:

Organizations store vast amounts of information across policies, technical manuals, FAQs, standard operating procedures (SOPs), project documentation, and knowledge repositories. Employees often struggle to locate accurate and up-to-date information, resulting in increased search time and inconsistent responses. Traditional Large Language Models (LLMs) may generate outdated or inaccurate answers because they rely only on their pre-trained knowledge.

Solution with AI:

This use case demonstrates how a Retrieval-Augmented Generation (RAG) system enables organizations to build an intelligent knowledge assistant by combining semantic search with Large Language Models. Instead of relying solely on the LLM's internal knowledge, the system

Copyright © 2026. All rights reserved

retrieves relevant information from an enterprise knowledge base and uses it to generate accurate, context-aware, and grounded responses.

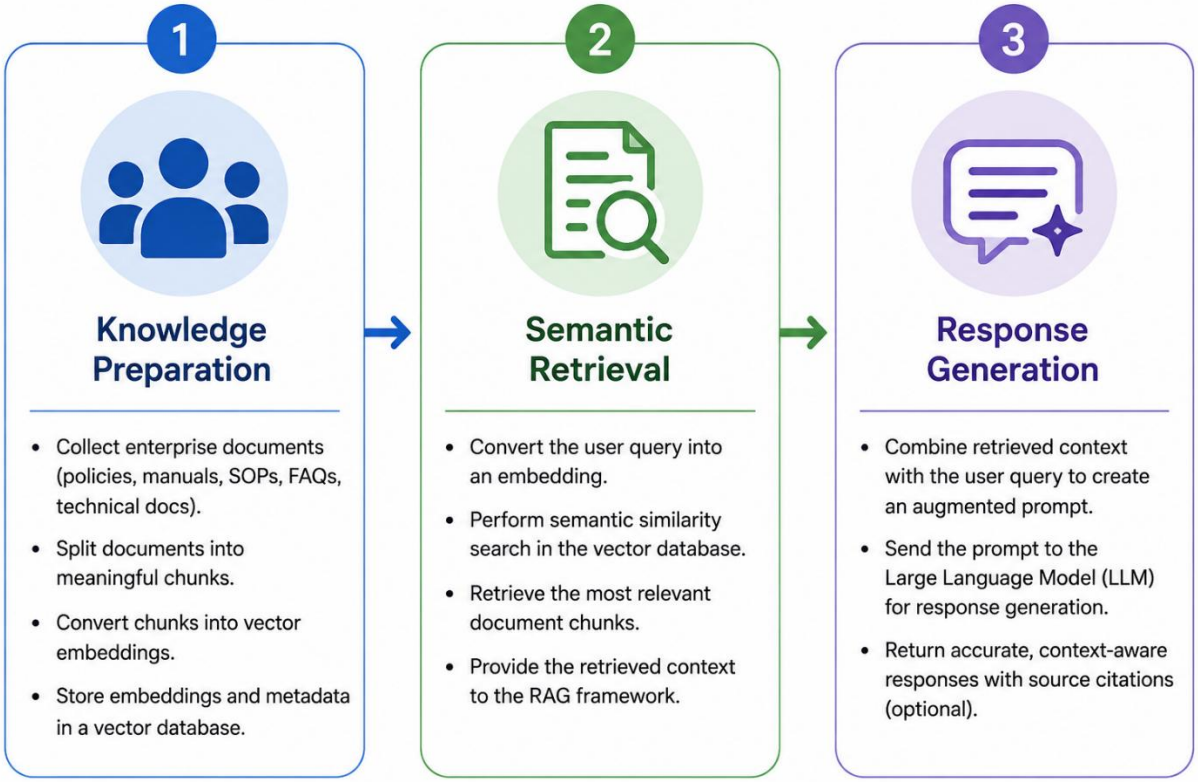


Figure. 8.4: Enterprise RAG System Workflow

Figure. 8.4 illustrates the end-to-end workflow of an enterprise RAG system as outlined below:

1. Knowledge Preparation

- Collect enterprise documents such as policies, manuals, SOPs, FAQs, and technical documentation.
- Split documents into meaningful chunks and convert them into vector embeddings.
- Store embeddings along with metadata in a vector database for semantic retrieval.

Copyright © 2026. All rights reserved

2. Semantic Retrieval

- Convert the user's query into an embedding.
- Perform semantic similarity search to retrieve the most relevant document chunks.
- Provide the retrieved context to the RAG framework for prompt augmentation.

3. Response Generation

- Combine the retrieved context with the user's query to create an augmented prompt.
- Send the prompt to the Large Language Model (LLM) for response generation.
- Return an accurate, context-aware response, optionally including citations to the source documents.

Benefits:

- Improved response accuracy with reduced hallucinations.
- Access to the latest organizational knowledge without retraining the LLM.
- Faster information retrieval through semantic search.
- Domain-specific responses tailored to enterprise knowledge.
- Enhanced productivity and better decision-making.

Example:

A healthcare organization deploys a RAG-based assistant to help doctors and staff access clinical guidelines, hospital policies, and treatment protocols. When a physician asks about

Copyright © 2026. All rights reserved

the latest patient discharge procedure, the assistant retrieves the most relevant policy documents from the hospital's knowledge base and generates a response based on the current approved guidelines, ensuring accurate and up-to-date information.

Building an enterprise knowledge assistant using RAG enables organizations to deliver accurate, explainable, and context-aware AI responses by integrating external knowledge retrieval with Large Language Models, while keeping the knowledge base continuously updated without retraining the model.

8.2 Vector databases and embeddings

Vector databases and embeddings form the foundation of Retrieval-Augmented Generation (RAG) systems. Embeddings convert textual information into numerical vectors that capture semantic meaning, while vector databases store and retrieve these vectors efficiently to enable fast and accurate semantic search. Let’s dive deeper into this in the following section.

Embeddings:

Embeddings are dense numerical vector representations of text, images, or other data generated using machine learning models. Unlike keyword-based representations, embeddings capture the semantic meaning and contextual relationships between pieces of information.

For example, the sentences:

- *"The patient has a fever."*
- *"The person is experiencing a high temperature."*

have different words but similar meanings. An embedding model generates vectors that are close together in the vector space, enabling semantic retrieval.

Copyright © 2026. All rights reserved

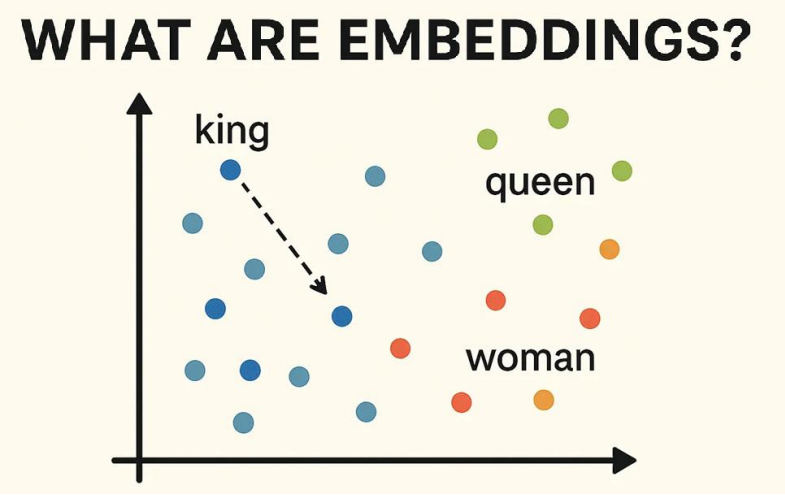


Figure. 8.5: A Simple Visual Representation of Embeddings

The image provides a simple visual representation of embeddings, which are numerical representations (vectors) of words, sentences, images, or other data that capture their meaning. Instead of representing words as text, machine learning models convert them into points in a high-dimensional vector space.

The proximity indicates semantic relationships:

- *King* and *Queen* are closely related because both represent royalty.
- *Queen* is closer to *Woman* than *King* because they share the concept of female gender.
- Unrelated words would be positioned much farther away.

Key Characteristics:

- Represent data as high-dimensional vectors.
- Capture semantic similarity instead of exact keywords.
- Generated using pre-trained embedding models.

Copyright © 2026. All rights reserved

- Used for semantic search, recommendation systems, clustering, and RAG.

How embeddings work?

Embeddings work by converting data (usually text) into vectors of numbers that capture semantic meaning. The model learns that words, sentences, or documents with similar meanings should have vectors that point in similar directions in a high-dimensional space.

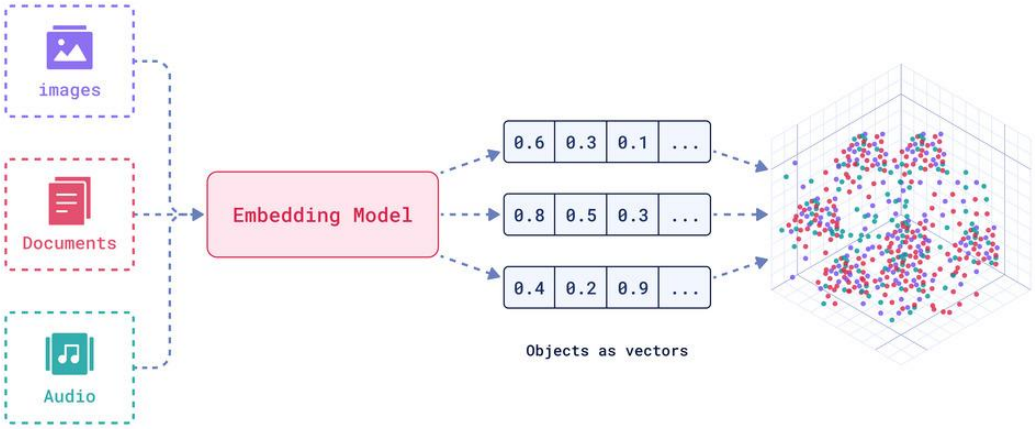


Figure. 8.5: Workflow of Vector Embeddings

The diagram illustrates the complete workflow of how an embedding model converts different types of data into vector embeddings that can be used for semantic search, clustering, recommendation systems, and Retrieval-Augmented Generation (RAG).

- **Multi-modal Input:** Images, documents, and audio are provided as input to the embedding model.

- **Embedding Model:** The model extracts semantic features and converts each input into a fixed-length numerical vector (embedding).

- **Vector Representation:** Each object is represented as a dense vector (e.g., [0.6, 0.3, 0.1, ...]) that captures its meaning.

Copyright © 2026. All rights reserved

- **Vector Space Mapping:** The embeddings are placed in a high-dimensional vector space.
- **Semantic Clustering:** Objects with similar meanings are positioned close together, while dissimilar objects are farther apart.
- **Similarity Search:** Distance metrics such as **Cosine Similarity**, **Euclidean Distance**, or **Dot Product** are used to identify the nearest vectors.

Did you know?
Two sentences with completely different words can have nearly identical embeddings if they convey the same meaning.

Quick Fact:
One embedding can represent a word, sentence, paragraph, document, image, audio clip, or code snippet.

Advantage of Vector Embeddings:

Embeddings offer several advantages over traditional keyword-based representations by capturing semantic meaning and improving the accuracy of AI-powered applications. The table below summarizes the key benefits of using embeddings.

Table 3: Advantages of Vector Embeddings

Advantage	Description
Semantic Search	Retrieves information based on meaning rather than exact keyword matches.

Copyright © 2026. All rights reserved

Contextual Understanding	Captures the contextual meaning of words, sentences, and documents.
Improved RAG Performance	Enhances retrieval accuracy by identifying the most relevant context for Large Language Models (LLMs).
Multilingual Support	Maps semantically similar content across different languages into a shared vector space.
Reduced Keyword Dependency	Finds relevant information even when different words or phrases express the same meaning.

Vector Databases:

A vector database is a specialized database designed to store, index, and retrieve vector embeddings efficiently. It performs similarity searches to find the most relevant vectors based on semantic closeness, making it an essential component of RAG systems.

Functions of a Vector Database:

A vector database is designed to efficiently store, index, and retrieve vector embeddings. Its specialized functions enable fast similarity search and support AI applications such as semantic search, RAG, and recommendation systems.

Copyright © 2026. All rights reserved

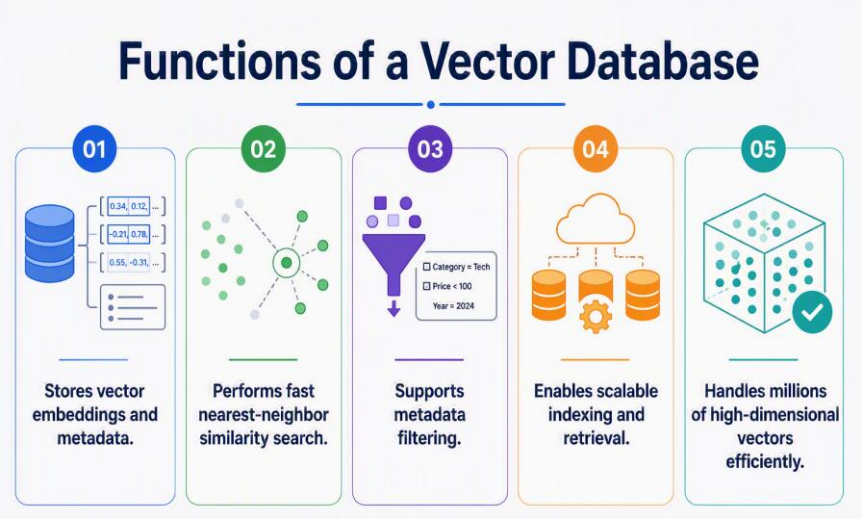


Figure. 8.6: Functions of a Vector Database

The diagram highlights the core functions of a vector database in managing and retrieving embedding vectors for AI applications.

- **Stores Vector Embeddings:** Stores high-dimensional vector embeddings along with their associated metadata for efficient organization and retrieval.
- **Performs Similarity Search:** Finds the most semantically similar vectors using similarity metrics such as Cosine Similarity, Euclidean Distance, or Dot Product.
- **Supports Metadata Filtering:** Refines search results by applying filters based on metadata such as category, source, date, or tags.
- **Enables Efficient Indexing and Retrieval:** Uses optimized indexing techniques, such as Approximate Nearest Neighbor (ANN), to perform fast vector searches.
- **Handles Large-Scale Vector Data:** Efficiently manages and retrieves millions or billions of high-dimensional vectors with low latency, making it suitable for enterprise-scale AI applications.

Copyright © 2026. All rights reserved

How vector database works?

A vector database stores and searches vector embeddings instead of traditional text or numeric records. It retrieves information based on semantic similarity rather than exact keyword matching.

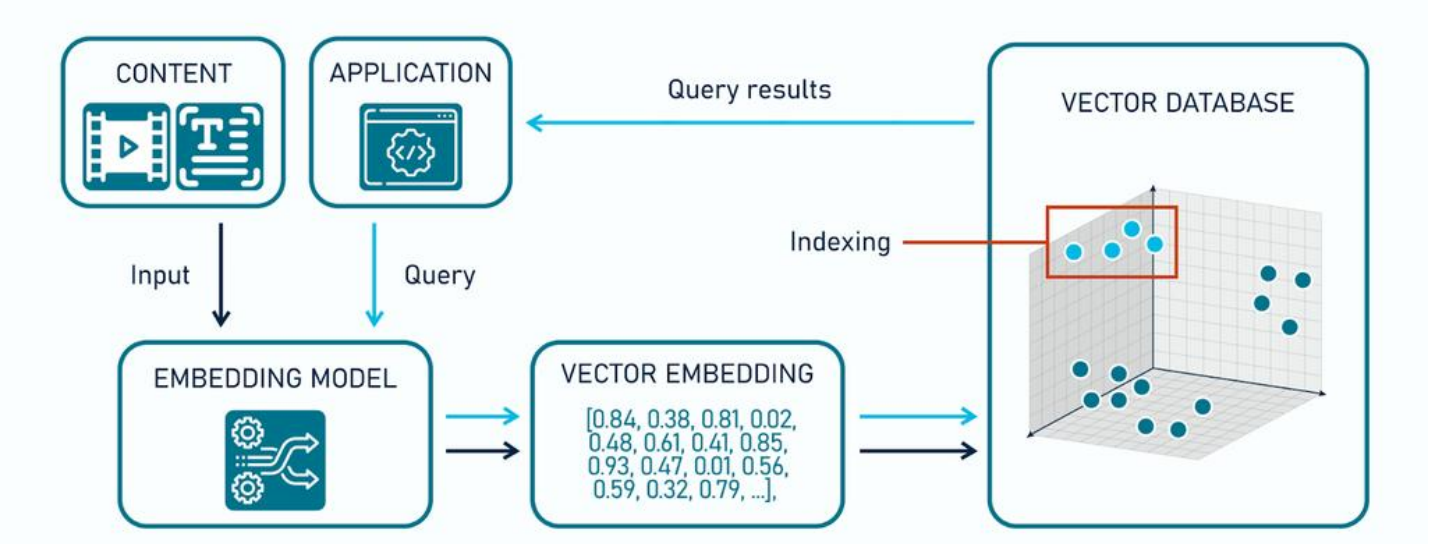


Figure. 8.7: Workflow of Vector Database

The figure illustrates the workflow of storing and retrieving information using a vector database.

- **Content Input:** Documents, images, videos, or other unstructured data are provided as input to the embedding model.
- **Embedding Generation:** The embedding model converts the input data into **vector embeddings** (fixed-length numerical representations) that capture semantic meaning.
- **Vector Storage and Indexing:** The generated embeddings are stored in the vector database, where they are organized using optimized indexing techniques (e.g., Approximate Nearest Neighbor) for efficient retrieval.

Copyright © 2026. All rights reserved

- **User Query:** When a user submits a query through an application, the query is also converted into a vector embedding using the same embedding model.

- **Similarity Search:** The vector database compares the query embedding with the indexed vectors using similarity metrics such as **Cosine Similarity**, **Euclidean Distance**, or **Dot Product**.

- **Query Results:** The database retrieves the most semantically similar vectors and returns the corresponding content to the application.

Did you know?
A vector database can search through **millions of embeddings in just a few milliseconds** using optimized indexing algorithms.

Popular Vector Databases:

Several vector databases are available today, each offering unique features for managing embeddings and supporting AI-powered search and retrieval applications. Let's discuss few of them.

1. Pinecone:

Pinecone is a fully managed, cloud-native vector database designed for storing, indexing, and searching high-dimensional vector embeddings. It eliminates infrastructure management, allowing developers to focus on building AI applications.

Key Features of Pinecone:

Pinecone offers a range of features that enable fast, scalable, and reliable vector storage and similarity search for production-grade AI applications.

Copyright © 2026. All rights reserved



Figure. 8.8: Key Features of Pinecone

The figure highlights the key features of Pinecone, a managed vector database designed for high-performance semantic search and Retrieval-Augmented Generation (RAG) applications.

Key Features of Pinecone are:

- **Scalability:** Efficiently handles millions to billions of vector embeddings with automatic scaling.
- **Cost Efficiency:** Fully managed infrastructure minimizes operational and maintenance costs.
- **Real-Time Operations:** Supports instant vector insertion, updates, and deletion for live applications.
- **Query Capabilities:** Provides fast similarity search, metadata filtering, and hybrid search for accurate retrieval.
- **Indexing Algorithms (HNSW):** Uses the Hierarchical Navigable Small World (HNSW) algorithm for high-speed, high-accuracy approximate nearest neighbor (ANN) search.

Copyright © 2026. All rights reserved

How Pinecone works?

Pinecone is a managed vector database that stores, indexes, and retrieves vector embeddings for fast semantic search. It enables AI applications to efficiently find relevant information and provide context-aware responses in RAG systems.

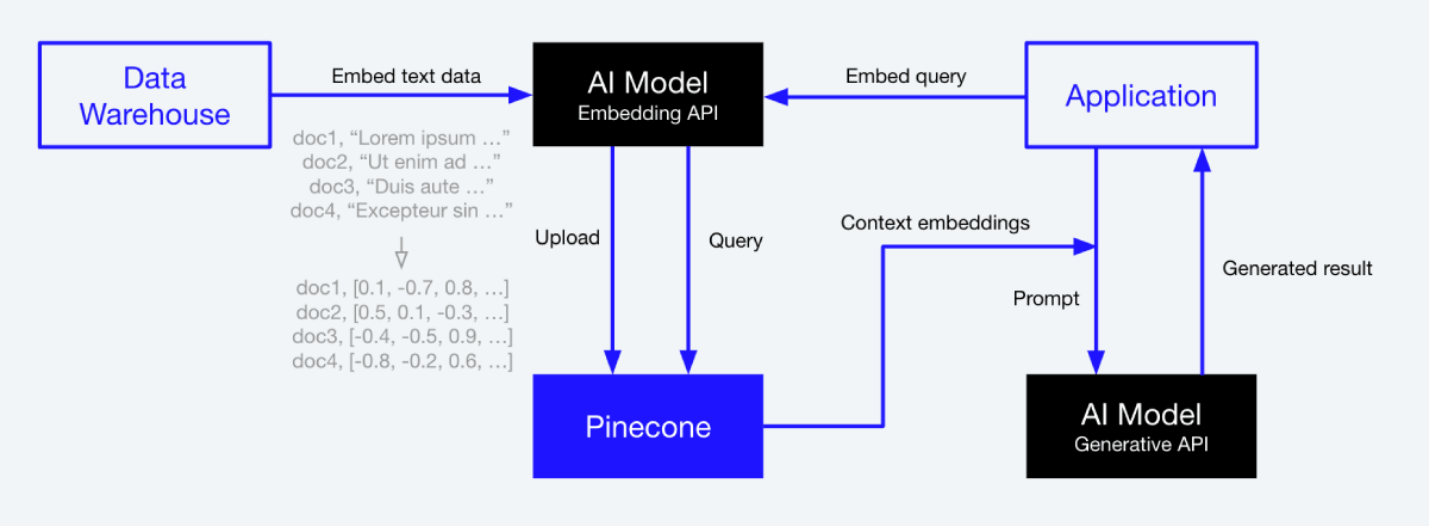


Figure. 8.9: Working of Pinecone in a RAG Pipeline

The figure presents working of Pinecone in a RAG pipeline. Key steps are:

- **Data Embedding:** Documents from the data warehouse are converted into vector embeddings using an embedding model.
- **Vector Storage:** The generated embeddings are uploaded and indexed in Pinecone for efficient similarity search.
- **Query Embedding:** When a user submits a query, the application converts it into a query embedding using the same embedding model.
- **Similarity Search:** Pinecone compares the query embedding with stored vectors and retrieves the most relevant context embeddings.

Copyright © 2026. All rights reserved

- **Context Injection:** The retrieved context is combined with the user's prompt and sent to the generative AI model.

- **Response Generation:** The generative AI model uses the retrieved context to produce an accurate, context-aware response, which is returned to the application.

Advantages of Pinecone:

Pinecone offers several advantages that make it a preferred vector database for modern AI and Retrieval-Augmented Generation (RAG) applications. The table below summarizes its key benefits and their significance.

Table 4: Advantages of Pinecone

Advantage	Description
No Infrastructure Management	Fully managed service eliminates the need to configure or maintain servers and databases.
Automatic Scalability	Seamlessly scales to accommodate growing datasets and increasing query workloads.
Easy LLM Integration	Integrates with frameworks such as LangChain and LlamaIndex for building RAG applications.
Production-Ready	Provides high availability, low latency, and reliability for enterprise-scale AI applications.

Copyright © 2026. All rights reserved

Did you know?
Pinecone is widely used in **Retrieval-Augmented Generation (RAG)** to provide relevant context to Large Language Models (LLMs).

Quick Fact:
Pinecone is best for production-ready, cloud-native AI applications with minimal operational overhead.

2. FAISS

FAISS (Facebook AI Similarity Search) is an open-source library developed by Meta for efficient similarity search and clustering of dense vectors. It is optimized for handling large-scale datasets on both CPUs and GPUs.

Key Features of FAISS:

FAISS provides high-performance indexing and similarity search capabilities, making it an efficient solution for managing and retrieving large-scale vector embeddings.

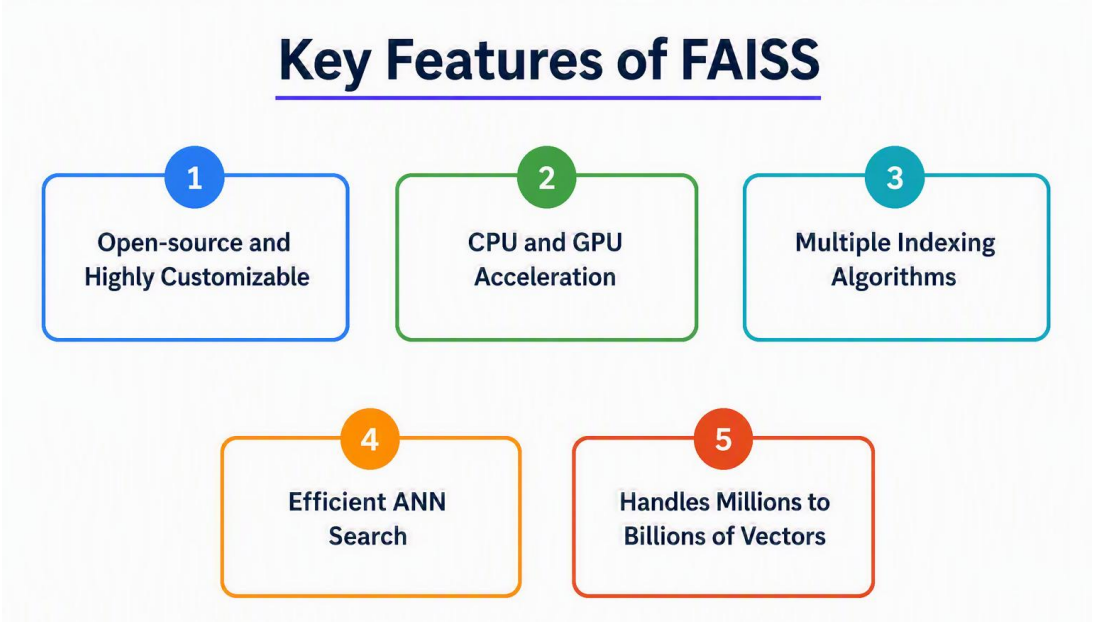


Figure. 8.10: Key Features of FAISS

Copyright © 2026. All rights reserved

The figure below highlights the key features that make FAISS a powerful and efficient library for large-scale vector indexing and similarity search.

Key Features of FAISS:

- **Open-Source and Highly Customizable:** Free, open-source library that can be customized for diverse similarity search applications.
- **CPU and GPU Acceleration:** Supports both CPU and GPU processing to deliver high-speed indexing and vector search.
- **Multiple Indexing Algorithms:** Offers various indexing methods such as Flat, IVF, HNSW, and PQ to balance speed, accuracy, and memory usage.
- **Efficient ANN Search:** Uses Approximate Nearest Neighbor (ANN) algorithms for fast and accurate similarity search.
- **Handles Millions to Billions of Vectors:** Efficiently stores and searches massive vector datasets, making it suitable for large-scale AI applications.

Working of FAISS:

FAISS performs efficient similarity search by converting data into vector representations, indexing them, and quickly retrieving the nearest vectors using optimized search algorithms.

Copyright © 2026. All rights reserved

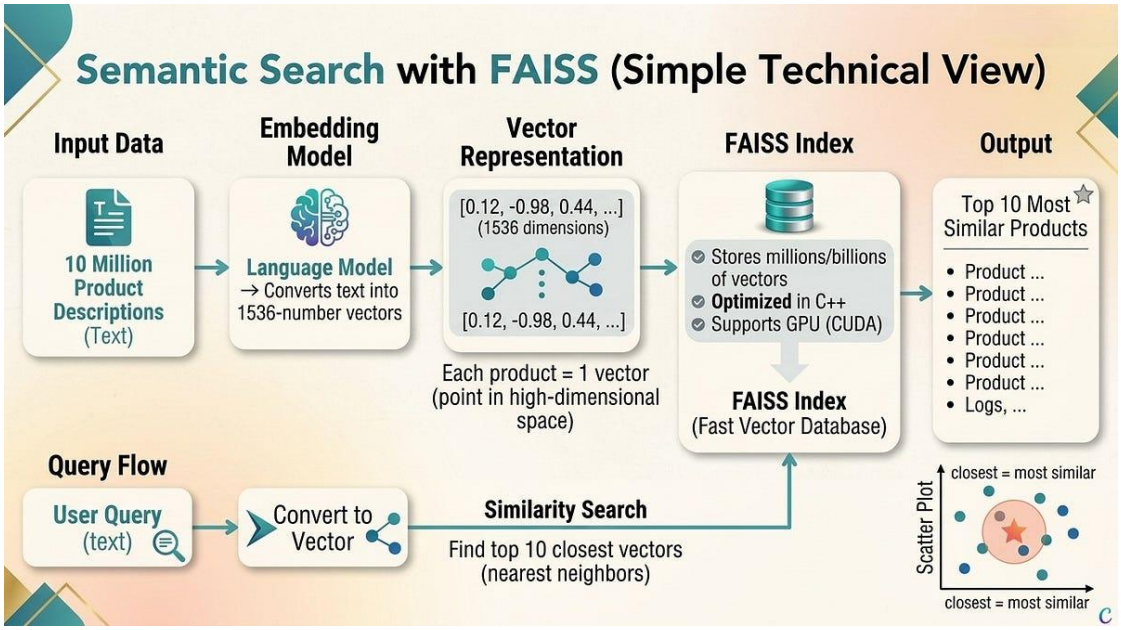


Figure. 8.11: Working of FAISS

The figure illustrates how FAISS performs semantic search by converting text into vector embeddings, indexing them, and retrieving the most similar vectors using efficient nearest neighbor search.

- **Input Data:** Text documents (e.g., product descriptions) are collected as the input dataset.

- **Embedding Generation:** An embedding model converts each document into a high-dimensional vector representation.

- **Vector Representation:** Each document is represented as a numerical vector that captures its semantic meaning.

- **FAISS Indexing:** The vectors are stored and indexed using FAISS for efficient similarity search. FAISS supports optimized CPU and GPU processing.

- **Query Embedding:** The user's search query is converted into a vector using the same embedding model.

Copyright © 2026. All rights reserved

- **Similarity Search:** FAISS compares the query vector with indexed vectors and identifies the nearest neighbors based on vector similarity.

- **Result Retrieval:** The most similar documents or products are returned as the search results.

Advantages of FAISS:

FAISS offers several advantages that make it a preferred library for high-performance vector search and large-scale AI applications. The table below summarizes its key benefits.

Table 5: Advantages of FAISS

Advantage	Description
Extremely Fast Similarity Search	Delivers high-speed nearest neighbor search for vector embeddings.
Excellent Performance for Large Datasets	Efficiently handles millions to billions of vectors with low latency.
Ideal for Research and Custom AI Systems	Provides flexibility for building and optimizing custom vector search solutions.
Supports Advanced Indexing Techniques	Offers multiple indexing methods (e.g., Flat, IVF, HNSW, PQ) to balance speed, accuracy, and memory usage.

Copyright © 2026. All rights reserved

Quick Fact:
FAISS is ideal for developers and researchers who need high-performance local vector search and full control over indexing.

3. Weaviate:

Weaviate is an open-source vector database that combines vector search with structured data management. It supports hybrid search, built-in AI modules, and GraphQL APIs, making it well suited for enterprise AI applications.

Key Features of Weaviate:

Weaviate provides AI-native features that enable efficient vector search, hybrid retrieval, and scalable management of vector embeddings for modern AI applications.

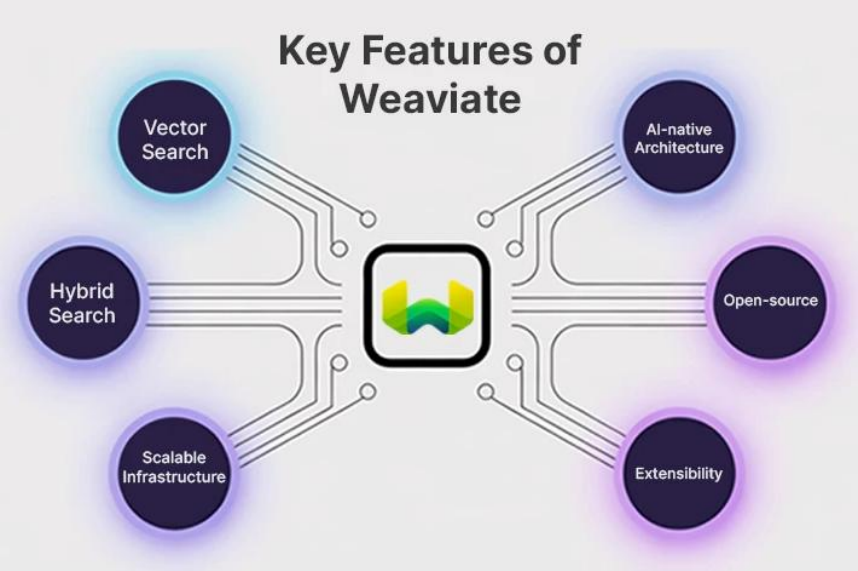


Figure. 8.12: Key Features of Weaviate

The figure below illustrates the key features of Weaviate that enable scalable, AI-native vector search and efficient retrieval for modern AI applications.

Copyright © 2026. All rights reserved

- **Vector Search:** Performs fast semantic search using vector embeddings to retrieve contextually relevant information.

- **Hybrid Search:** Combines keyword-based (BM25) and vector search for improved search accuracy and relevance.

- **Scalable Infrastructure:** Supports distributed deployment and scales efficiently to handle large datasets and high query volumes.

- **AI-Native Architecture:** Designed specifically for AI applications, with built-in support for embeddings, RAG, and machine learning workflows.

- **Open-Source:** Freely available and highly customizable, enabling flexible deployment in cloud or on-premises environments.

- **Extensibility:** Supports custom modules, APIs, and integrations with embedding models, LLMs, and AI frameworks such as LangChain and LlamaIndex.

Working of Weaviate:

Weaviate is an AI-native vector database that stores and indexes vector embeddings to enable fast semantic search and Retrieval-Augmented Generation (RAG). It supports both built-in and external embedding models for scalable AI applications.

Copyright © 2026. All rights reserved

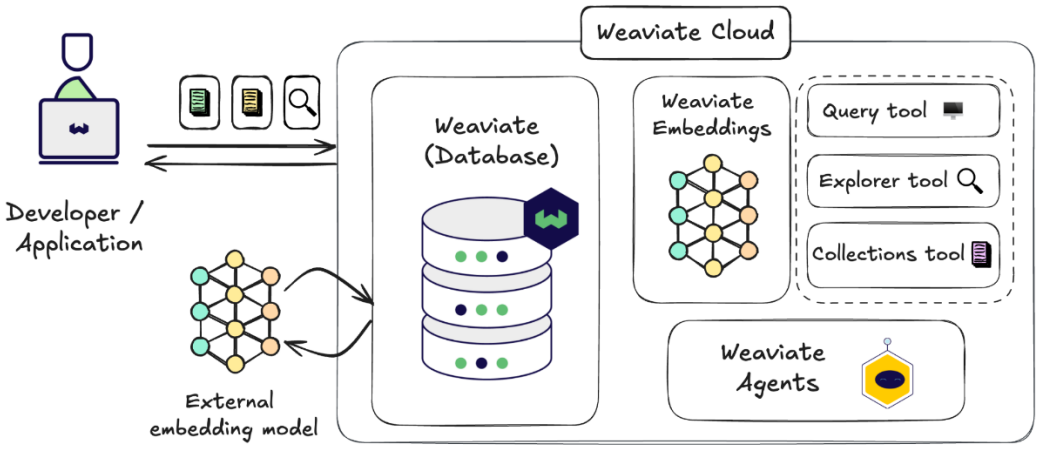


Figure. 8.13: Working of Weaviate

The figure illustrates how Weaviate manages vector embeddings and enables semantic search through its AI-native architecture.

- **Data Ingestion:** The developer or application uploads documents or data to Weaviate Cloud.
- **Embedding Generation:** Data is converted into vector embeddings using either Weaviate's built-in embedding models or external embedding models.
- **Vector Storage:** The generated embeddings are stored and indexed in the Weaviate database along with associated metadata.
- **Query Processing:** User queries are converted into vector embeddings using the same embedding model.
- **Similarity Search:** Weaviate performs semantic or hybrid search to retrieve the most relevant vectors based on similarity.
- **Result Access:** Retrieved results are accessed through query, explorer, or collections tools and returned to the application.

Copyright © 2026. All rights reserved

- **AI Integration:** Weaviate Agents and APIs enable seamless integration with AI applications, RAG pipelines, and LLM-based systems.

Advantages of Weaviate:

Weaviate offers several advantages that make it a powerful vector database for semantic search, RAG, and AI-driven applications. The table below summarizes its key benefits.

Table 6: Advantages of Weaviate

Advantage	Description
Combines Semantic and Keyword Search	Supports hybrid search by combining vector similarity with keyword-based retrieval for improved accuracy.
Easy Integration with Machine Learning Models	Integrates seamlessly with embedding models, LLMs, and AI frameworks such as LangChain and LlamaIndex.
Supports Multimodal Data	Stores and searches multiple data types, including text, images, audio, and other multimodal content.
Flexible Deployment Options	Can be deployed on cloud, on-premises, or Kubernetes environments to meet different application needs.

Copyright © 2026. All rights reserved

Quick Fact:
Weaviate is well suited for enterprise AI applications that require hybrid search, structured metadata, and flexible deployment options.

Role of Embeddings and Vector Databases in RAG:

Embeddings and vector databases are the core components that enable semantic retrieval in Retrieval-Augmented Generation (RAG). Embeddings capture the meaning of text as numerical vectors, while vector databases efficiently store and retrieve these vectors to provide relevant context for the Large Language Model (LLM).

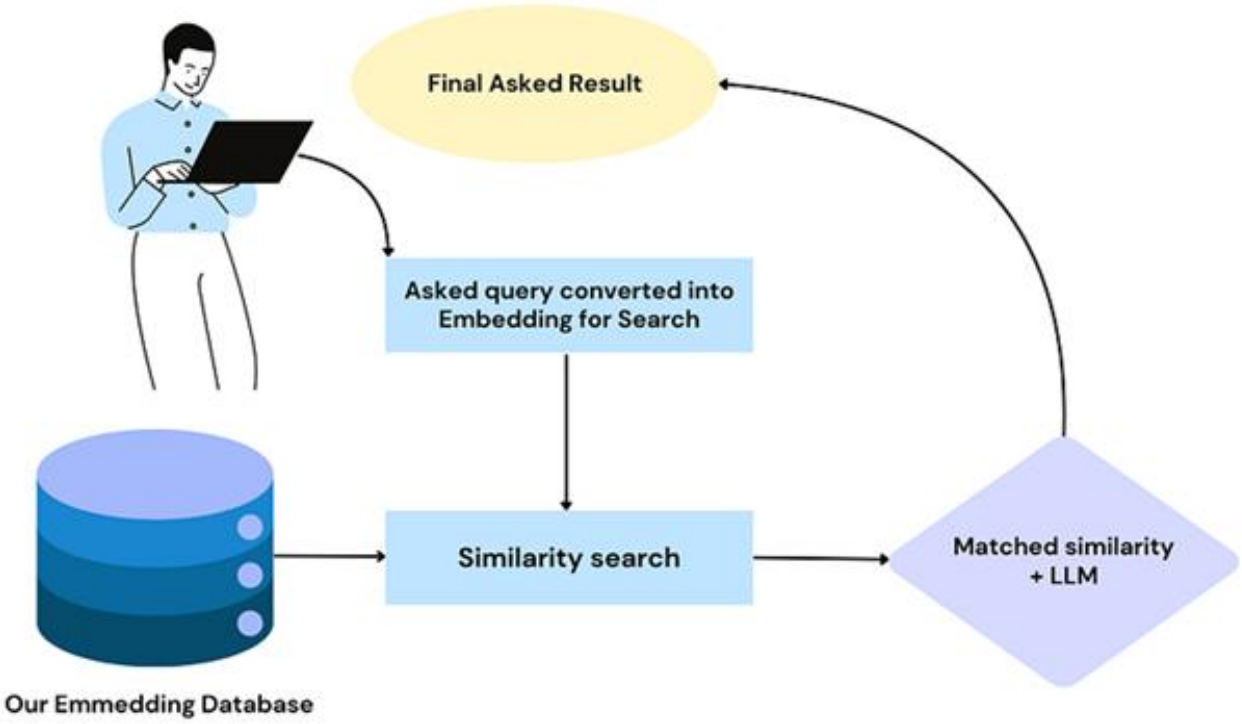


Figure. 8.14: Role of Embeddings and Vector Databases in RAG

The figure illustrates how embeddings and a vector database work together to retrieve relevant information and generate accurate responses in a Retrieval-Augmented Generation (RAG) system.

Copyright © 2026. All rights reserved

1. **Step 1: User Query**

   - The user submits a query to the RAG system.

2. **Step 2: Query Embedding**

   - The query is converted into a numerical vector (embedding) using an embedding model.

3. **Step 3: Similarity Search**

   - The embedding is compared with vectors stored in the vector database.

   - A similarity search identifies the most semantically relevant document embeddings.

4. **Step 4: Context Retrieval and LLM Processing**

   - The matched document chunks are retrieved from the vector database.

   - The retrieved context is provided to the Large Language Model (LLM), which generates a context-aware response.

5. **Step 5: Final Response**

   - The RAG system returns the final, accurate, and grounded response to the user based on the retrieved information.

Copyright © 2026. All rights reserved

Use Case

Title: Semantic Product Search Using Vector Databases and Embeddings

Scenario:

An e-commerce company offers thousands of products across multiple categories. Customers often use natural language queries such as *"comfortable running shoes for long-distance jogging"* instead of exact product names. Traditional keyword-based search fails to understand the intent behind such queries, resulting in irrelevant search results and poor customer experience.

Solution with AI:

This use case demonstrates how embeddings and a vector database enable semantic product search. Product descriptions are converted into vector embeddings and stored in a vector database. When a customer submits a query, it is also converted into an embedding. The system performs a similarity search to retrieve products that are semantically related to the query, and the retrieved results are provided to the LLM to generate personalized product recommendations.

Copyright © 2026. All rights reserved

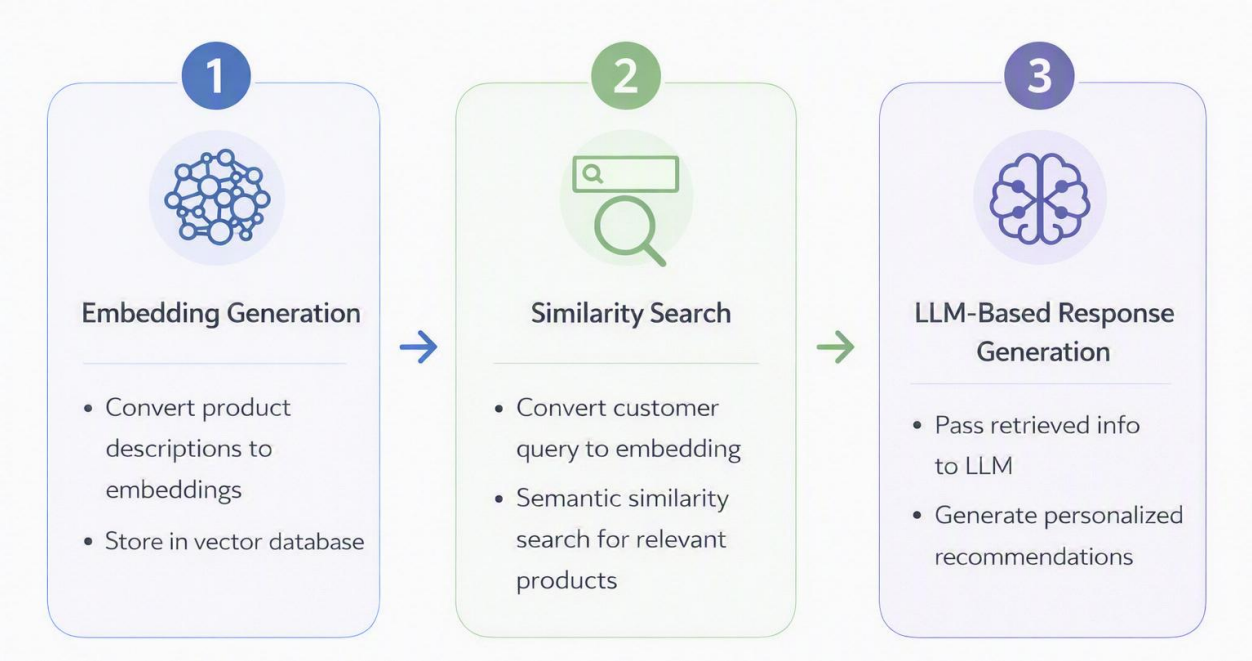


Figure. 8.15: Semantic Search Using Embeddings and Vector Database

Figure. 8.15 illustrates the semantic search workflow as outlined below:

1. Embedding Generation

- Convert product descriptions into vector embeddings.
- Store embeddings and metadata in a vector database.

2. Similarity Search

- Convert the customer's query into an embedding.
- Perform semantic similarity search to retrieve the most relevant products.

3. LLM-Based Response Generation

- Pass the retrieved product information to the LLM.
- Generate personalized and context-aware product recommendations.

Copyright © 2026. All rights reserved

Benefits:

- Retrieves products based on semantic meaning rather than keywords.
- Improves search accuracy and recommendation quality.
- Supports natural language queries.
- Delivers faster and more relevant search results.
- Enhances customer satisfaction and engagement.

Example:

A customer searches for "lightweight laptop for programming with long battery life." Instead of matching only the keyword *"laptop,"* the system retrieves products that best satisfy the user's intent based on semantic similarity. The LLM then summarizes the top recommendations, highlighting features such as processor, RAM, battery life, and price.

Using embeddings and vector databases enables intelligent semantic search by understanding the meaning behind user queries, resulting in more accurate retrieval and personalized AI-generated responses.

8.3 Advanced RAG patterns

As Retrieval-Augmented Generation (RAG) applications grow in complexity, basic retrieval techniques may not always provide the most relevant context. Advanced RAG patterns enhance retrieval quality by improving document segmentation, query understanding, and knowledge representation, resulting in more accurate, context-aware, and reliable AI-generated responses. Let’s explore common advanced RAG patterns.

Copyright © 2026. All rights reserved

1. Advanced Chunking:

Advanced Chunking is the process of intelligently dividing large documents into meaningful and context-preserving chunks instead of using fixed-size text segments. Better chunking improves retrieval accuracy by ensuring that relevant information remains together.

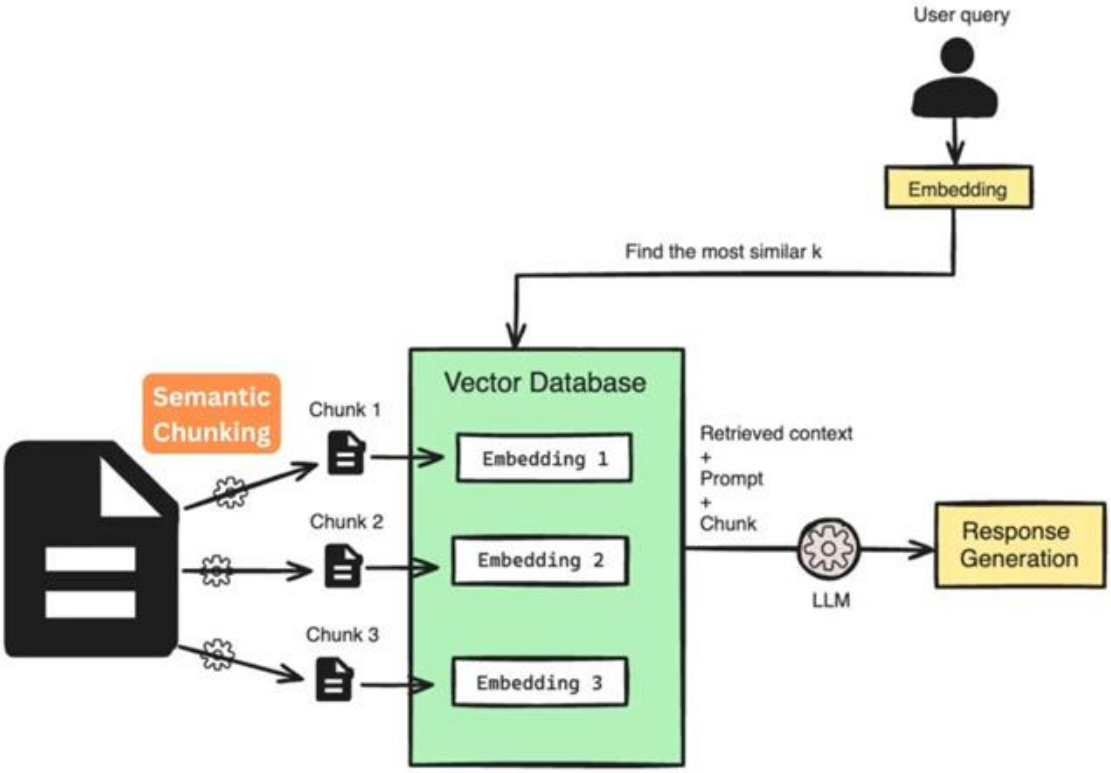


Figure. 8.16: Advanced Chunking in RAG Applications

The figure illustrates how advanced chunking improves information retrieval in a Retrieval-Augmented Generation (RAG) system by dividing documents into meaningful chunks before storing them in a vector database.

- **Semantic Chunking**: The document is divided into meaningful chunks that preserve contextual information.
- **Embedding Storage**: Each chunk is converted into a vector embedding and stored in the vector database.

Copyright © 2026. All rights reserved

- **Query Embedding**: The user's query is converted into an embedding for semantic search.

- **Similarity Search**: The vector database retrieves the top-k most relevant chunks based on semantic similarity.

- **Response Generation**: The retrieved chunks are combined with the prompt and passed to the LLM, which generates a context-aware response.

Why is advanced chunking needed?

Traditional fixed-size chunking may:

- Split related information across multiple chunks.

- Lose contextual relationships.

- Reduce retrieval accuracy.

- Return incomplete answers.

Advanced chunking overcomes these limitations by preserving semantic meaning.

Types of Advanced Chunking:

Advanced chunking techniques improve retrieval quality by creating context-rich and meaningful document segments. The table below outlines the commonly used chunking methods and their benefits.

Copyright © 2026. All rights reserved

Table 7: Types of Advanced Chunking

Advanced Chunking Type	Description
Semantic Chunking	Splits documents based on semantic meaning instead of fixed character or token count, grouping related sentences into coherent chunks.
Recursive Chunking	Breaks documents hierarchically (sections → paragraphs → sentences) while maintaining the document structure.
Sliding Window Chunking	Generates overlapping chunks so that adjacent chunks share common content.
Metadata-Aware Chunking	Associates each chunk with metadata such as title, author, page number, section, or source.

Quick Fact:
Semantic chunking can significantly improve retrieval accuracy by grouping text based on meaning rather than fixed size.

Did you know?
The effectiveness of a RAG system often depends more on **chunk quality** than on chunk quantity.

Copyright © 2026. All rights reserved

Advantages of Advanced Chunking:

Advanced chunking techniques improve the quality of retrieved information by creating meaningful and context-aware document chunks.

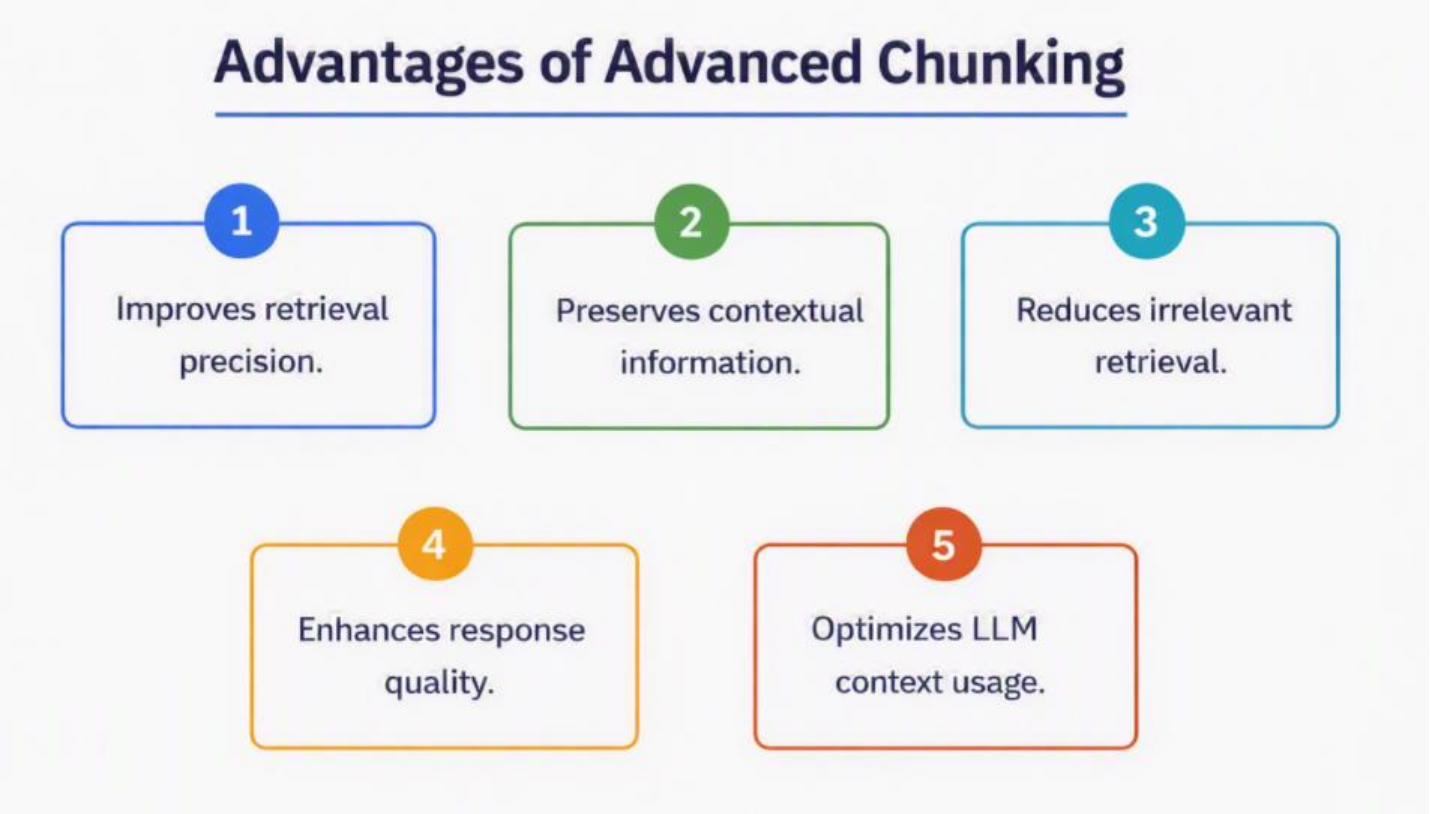


Figure. 8.17: Advantages of Advanced Chunking

The figure highlights the key benefits of advanced chunking techniques in Retrieval-Augmented Generation (RAG) systems.

Key benefits are:

- **Improves Retrieval Precision:** Creates meaningful chunks that increase the likelihood of retrieving relevant information.
- **Preserves Contextual Information:** Maintains the relationship between sentences and ideas, reducing context loss.

Copyright © 2026. All rights reserved

- **Reduces Irrelevant Retrieval:** Minimizes retrieval of unrelated or noisy content by generating semantically coherent chunks.

- **Enhances Response Quality:** Provides the LLM with richer and more relevant context, resulting in accurate and coherent responses.

- **Optimizes LLM Context Usage:** Efficiently utilizes the model's context window by supplying only the most relevant information, improving overall performance.

2. HyDE (Hypothetical Document Embeddings)

HyDE (Hypothetical Document Embeddings) is an advanced retrieval technique in which the LLM first generates a hypothetical answer or document for the user's query. This generated text is then converted into an embedding and used to retrieve semantically similar documents from the vector database.

Instead of embedding the user query directly, HyDE embeds the generated hypothetical document, often improving retrieval for ambiguous or complex queries.

Why is HyDE needed?

HyDE is used to improve retrieval quality when a user's query is short, ambiguous, or lacks sufficient context. Instead of directly searching with the original query, an LLM first generates a **hypothetical answer or document**, which is then converted into an embedding. This richer embedding is used to retrieve more relevant documents from the vector database.

Need for HyDE:

- Improves retrieval for vague or short queries.

- Captures semantic intent beyond exact keywords.

Copyright © 2026. All rights reserved

- Increases retrieval accuracy by generating context-rich embeddings.
- Reduces irrelevant search results in vector databases.
- Enhances RAG performance, leading to more informative and accurate LLM responses.

How HyDE works?

HyDE enhances document retrieval by generating a hypothetical answer before performing semantic search.

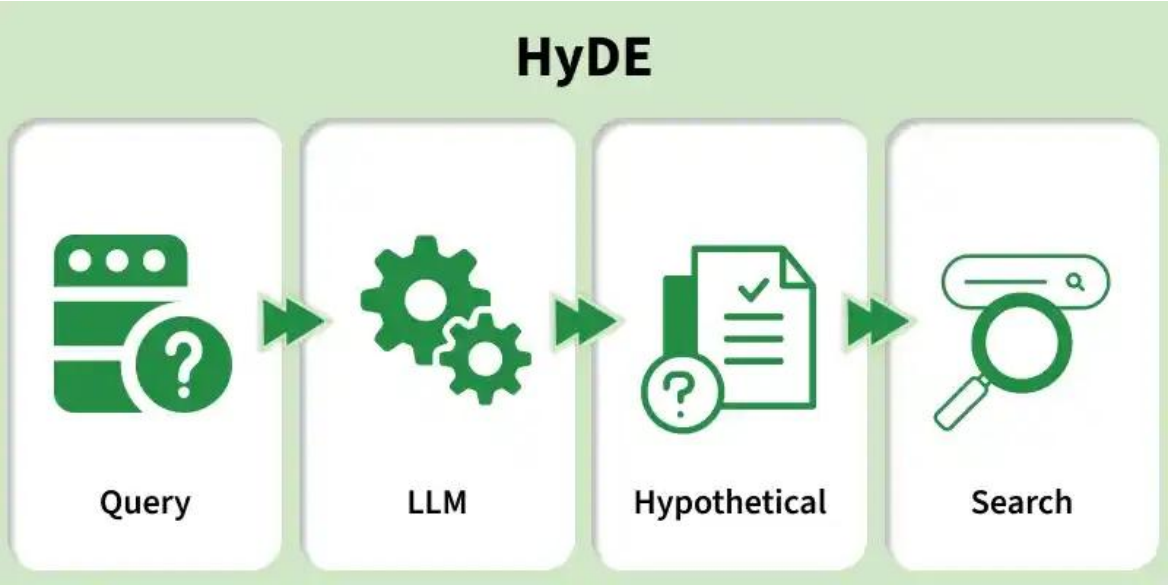


Figure. 8.18: Working of HyDE

The figure illustrates the HyDE workflow, where an LLM generates a hypothetical document from the user's query before performing semantic search. Instead of directly embedding the original query, the generated document is embedded and used to retrieve the most relevant information from the vector database, resulting in improved retrieval accuracy.

Copyright © 2026. All rights reserved

Workflow Steps

1. **Query:** The user submits a question or search query.

2. **LLM:** The Large Language Model generates a hypothetical document or answer based on the query.

3. **Hypothetical Document:** The generated document is converted into a vector embedding that better captures the query's semantic intent.

4. **Search:** The embedding is used to perform similarity search in the vector database, retrieving the most relevant documents for the final response.

Advantages of HyDE:

HyDE (Hypothetical Document Embeddings) improves retrieval quality by generating a hypothetical answer before performing similarity search. This approach helps retrieve semantically relevant documents even when the user's query is short, vague, or lacks sufficient context.

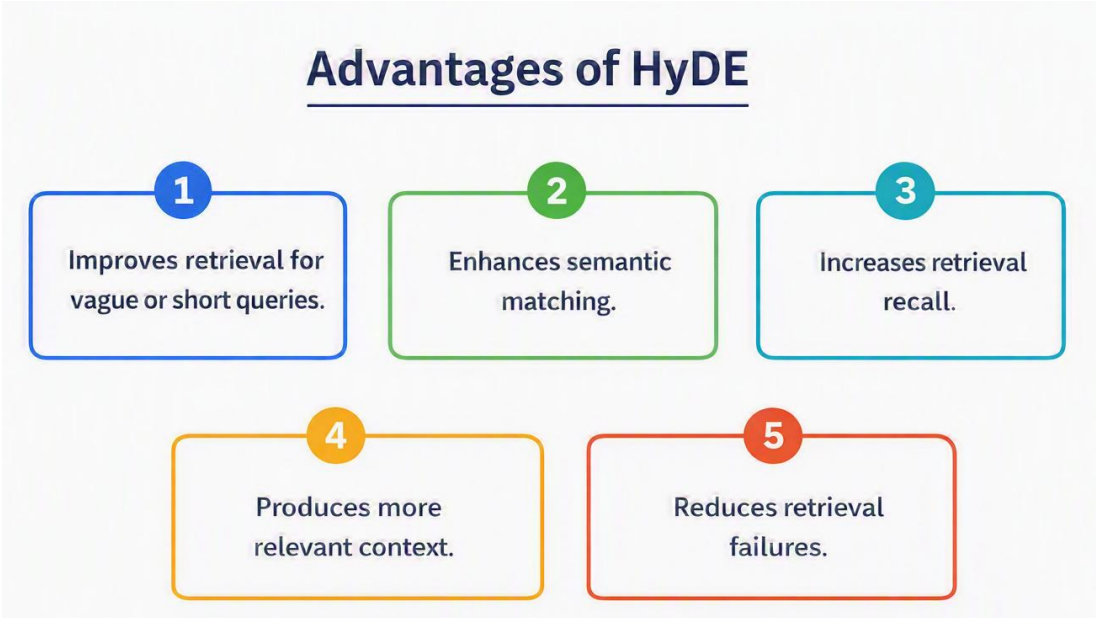


Figure. 8.19: Advantages of HyDE

Copyright © 2026. All rights reserved

The figure illustrates the key advantages of HyDE (Hypothetical Document Embeddings) in improving document retrieval for RAG systems.

- **Improves retrieval for vague or short queries:** Expands ambiguous queries into richer representations for better search.
- **Enhances semantic matching:** Retrieves documents based on meaning rather than exact keywords.
- **Increases retrieval recall:** Finds a larger set of relevant documents, even with limited query information.
- **Produces more relevant context:** Supplies the LLM with higher-quality retrieved content for accurate responses.
- **Reduces retrieval failures:** Minimizes irrelevant or empty search results, improving overall retrieval reliability.

Did You Know?
HyDE often retrieves relevant documents even when they **do not contain the exact query keywords**.

3. GraphRAG:

GraphRAG extends traditional RAG by incorporating knowledge graphs to capture relationships between entities such as people, organizations, documents, products, and events. Instead of retrieving isolated document chunks, GraphRAG retrieves connected information based on relationships, enabling deeper reasoning and multi-hop question answering.

Copyright © 2026. All rights reserved

How GraphRAG works?

GraphRAG enhances traditional RAG by organizing documents into a knowledge graph of entities and relationships, enabling the retrieval of structured and connected information. The retrieved graph context is combined with the user query to help the LLM generate more accurate, context-aware, and explainable responses.

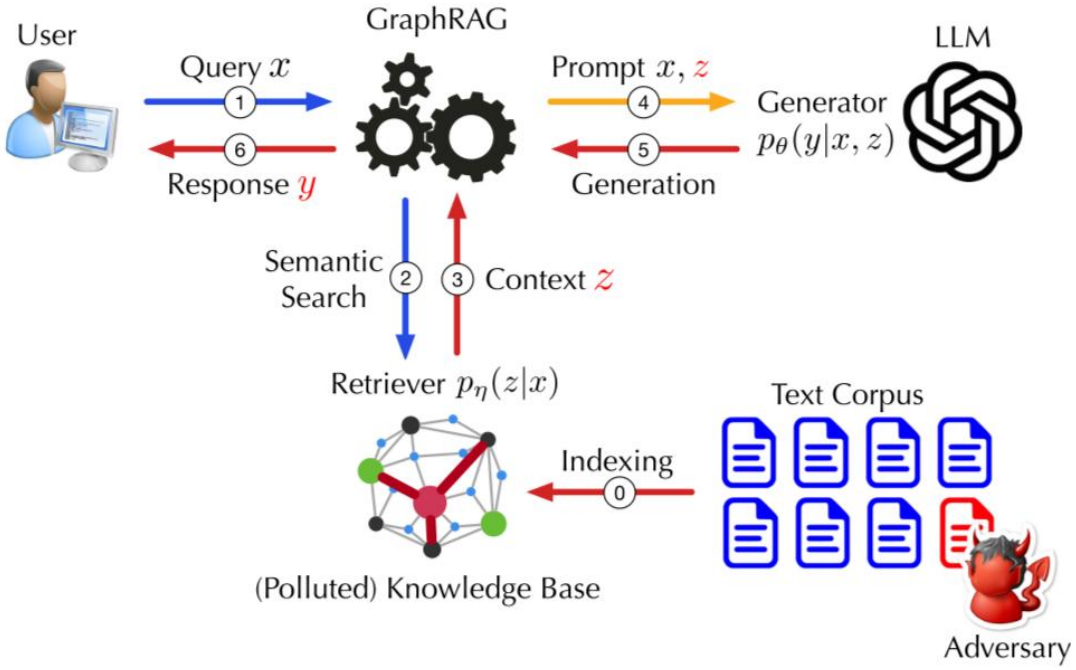


Figure. 8.20: Working of GraphRAG

The figure illustrates the GraphRAG query processing pipeline, where structured knowledge is retrieved from a knowledge graph before generating a response using an LLM.

- **Indexing:** Documents from the text corpus are converted into a **knowledge graph** containing entities and relationships. Malicious or noisy documents may pollute the knowledge base.

- **User Query:** The user submits a query (**x**) to the GraphRAG system.

- **Semantic Retrieval:** GraphRAG performs semantic search over the knowledge graph to retrieve the most relevant graph context (**z**).

Copyright © 2026. All rights reserved

- **Prompt Construction:** The retrieved context is combined with the user query to create an enriched prompt.

- **LLM Generation:** The prompt is passed to the LLM, which generates a context-aware response (**y**).

- **Response Delivery:** The generated response is returned to the user.

Advantages of GraphRAG over Traditional RAG:

While Traditional RAG relies on semantic similarity, GraphRAG enhances retrieval by incorporating entity relationships and graph structures. The following table compares their major advantages.

Table 8: Advantages of GraphRAG over Traditional RAG

GraphRAG	Traditional RAG
Captures relationships between entities using a knowledge graph.	Retrieves independent text chunks based on semantic similarity.
Supports multi-hop reasoning across connected information.	Limited reasoning across multiple documents.
Provides richer contextual understanding for complex queries.	May miss context spread across different documents.

Copyright © 2026. All rights reserved

Generates more explainable responses using graph paths and relationships.	Retrieval process is less transparent and harder to explain.
Reduces ambiguity by leveraging structured knowledge.	More prone to irrelevant or incomplete retrieval.
Improves retrieval accuracy for interconnected information.	Accuracy depends mainly on embedding similarity.
Handles complex enterprise knowledge bases more effectively.	Performs best on straightforward text retrieval tasks.
Suitable for domains such as healthcare, finance, legal, and scientific research.	Better suited for general-purpose document retrieval.

Did you know?
GraphRAG can answer questions that require connecting information across multiple documents through graph relationships.

Copyright © 2026. All rights reserved

Use Case

Title: Intelligent Research Assistant Using Advanced RAG Patterns

Scenario:

A research organization maintains thousands of scientific papers, technical reports, patents, and publications. Researchers often ask complex questions that require information from multiple documents and relationships between concepts. Traditional RAG systems may retrieve incomplete or less relevant information due to fixed-size chunking and limited contextual understanding.

Solution with AI:

This use case demonstrates how Advanced RAG Patterns—Advanced Chunking, HyDE, and GraphRAG—improve retrieval quality and reasoning. Semantic chunking preserves contextual information, HyDE enhances retrieval for ambiguous queries by generating hypothetical document embeddings, and GraphRAG leverages knowledge graphs to connect related entities, enabling accurate and comprehensive responses.

Copyright © 2026. All rights reserved

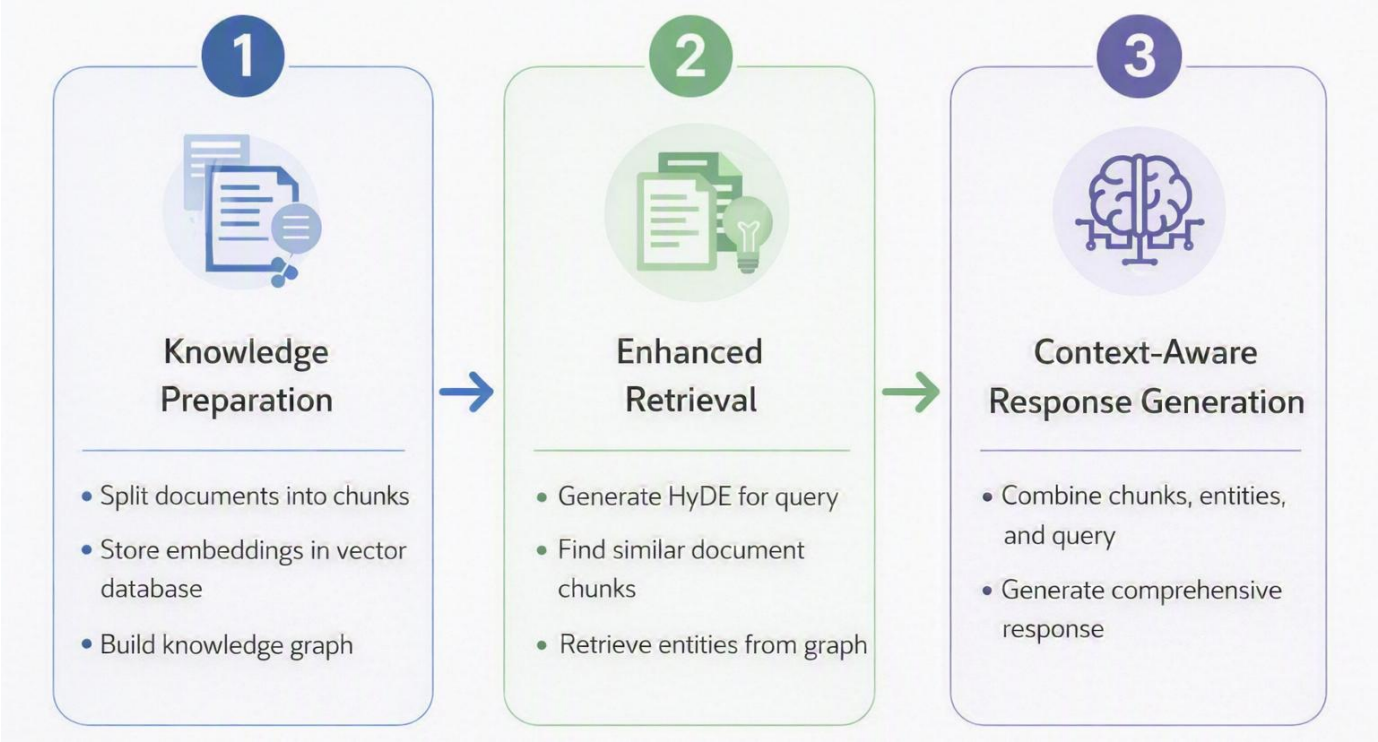


Figure. 8.22: Advanced RAG Workflow

Figure. 8.22 illustrates the advanced RAG process as outlined below:

1. Knowledge Preparation

- Split documents using semantic or recursive chunking.
- Generate embeddings and store them in a vector database.
- Build a knowledge graph to capture relationships among entities.

2. Enhanced Retrieval

- Generate a hypothetical document embedding (HyDE) for the user query.
- Perform semantic similarity search to retrieve the most relevant document chunks.
- Retrieve related entities and relationships from the knowledge graph.

Copyright © 2026. All rights reserved

3. Context-Aware Response Generation

- Combine retrieved chunks, graph relationships, and the user query.
- Generate a comprehensive, accurate, and explainable response using the LLM.

Benefits:

- Improves retrieval accuracy for complex and ambiguous queries.
- Preserves document context through intelligent chunking.
- Supports multi-hop reasoning using knowledge graphs.
- Reduces hallucinations and irrelevant retrieval.
- Delivers more accurate and explainable AI-generated responses.

Example:

A pharmaceutical researcher asks, "Which proteins are associated with Alzheimer's disease, and what recent studies discuss their role?" The system uses Semantic Chunking to retrieve relevant research sections, HyDE to improve retrieval for the complex query, and GraphRAG to identify relationships between proteins, diseases, and published studies. The LLM then generates a comprehensive response with connected insights from multiple sources.

Advanced RAG patterns enhance Retrieval-Augmented Generation by improving document retrieval, preserving contextual information, and enabling relationship-aware reasoning, resulting in more accurate and reliable AI-assisted knowledge discovery.

Copyright © 2026. All rights reserved

8.4 Evaluation metrics for RAG systems

Evaluating a Retrieval-Augmented Generation (RAG) system involves measuring both the quality of retrieved information and the correctness of the generated response. Modern evaluation frameworks such as RAGAS provide automated metrics that assess retrieval effectiveness and answer quality without requiring manually labeled datasets. Let's explore this in the below section.

RAGAS (Retrieval-Augmented Generation Assessment):

RAGAS is an open-source evaluation framework specifically designed for RAG systems. It evaluates the retrieval and generation components by using LLMs and embedding models instead of relying solely on human-annotated reference answers.

Key Features:

RAGAS (Retrieval-Augmented Generation Assessment) provides a comprehensive framework for evaluating the quality and effectiveness of RAG systems. It measures multiple aspects of retrieval and generation to ensure responses are accurate, relevant, faithful, and context-aware.

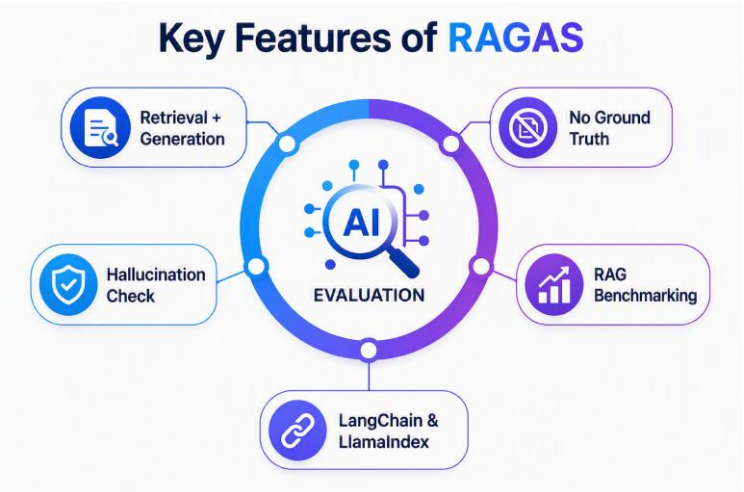


Figure. 8.23: Key Features of RAGAS

Copyright © 2026. All rights reserved

The figure highlights the five core capabilities of RAGAS for evaluating Retrieval-Augmented Generation (RAG) systems.

- **Retrieval + Generation:** Evaluates both the quality of retrieved documents and the generated response.
- **No Ground Truth:** Assesses RAG performance without requiring manually labeled reference answers.
- **RAG Benchmarking:** Enables objective comparison of different RAG models, retrievers, and configurations.
- **LangChain & LlamaIndex Integration:** Integrates seamlessly with popular RAG frameworks for easy evaluation.
- **Hallucination Check:** Detects unsupported or fabricated information by verifying response faithfulness to the retrieved context.
    - Evaluates both retrieval and generation quality.
    - Works without requiring ground-truth answers in many cases.
    - Detects hallucinations and unsupported claims.
    - Supports automated benchmarking of RAG pipelines.
    - Integrates with frameworks such as LangChain and LlamaIndex.

Copyright © 2026. All rights reserved

RAG Evaluation Metrices:

RAG evaluation metrics measure the effectiveness of a Retrieval-Augmented Generation (RAG) system by assessing the quality of both retrieved information and generated responses. These metrics help ensure that responses are accurate, relevant, and grounded in the retrieved context.

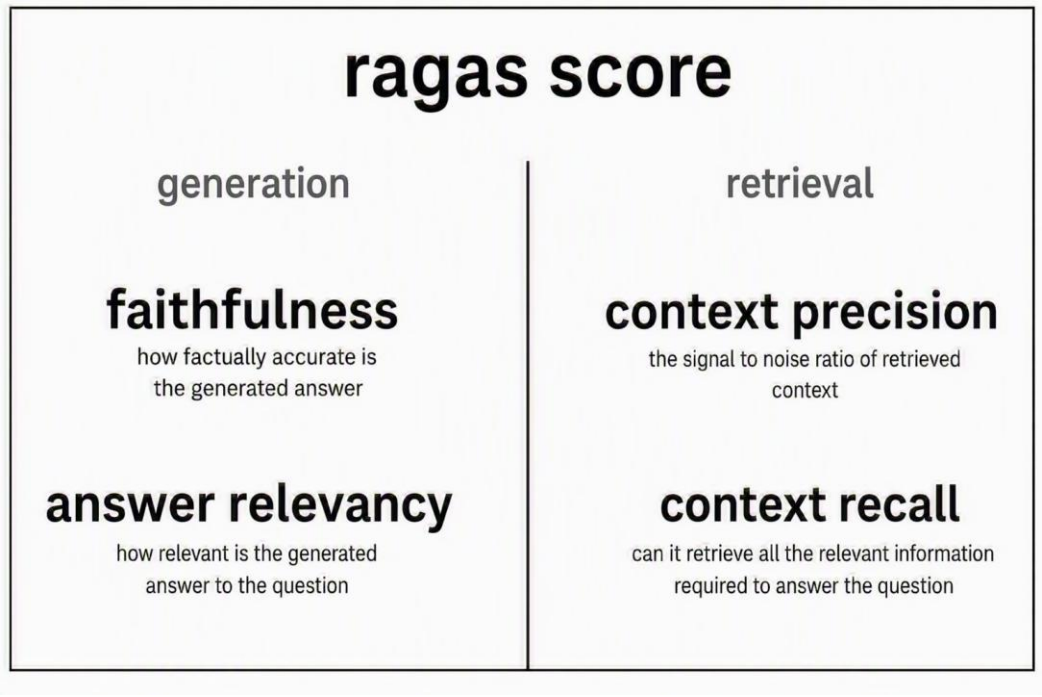


Figure. 8.24: RAG Evaluation Metrices

The figure shows that RAGAS evaluates a RAG system using four key metrics, grouped into generation and retrieval performance.

1. **Faithfulness:** measures whether the generated answer is fully supported by the retrieved context.

Purpose:

- Detects hallucinations.
- Ensures responses are grounded in retrieved documents.

Copyright © 2026. All rights reserved

- Verifies factual consistency.

High Score Indicates:

- The answer is based only on retrieved information.
- No unsupported or fabricated facts are included.

2. Answer Relevance: measures how well the generated response answers the user's query.

Purpose:

- Ensures responses remain focused on the user's question.
- Detects irrelevant or incomplete answers.

High Score Indicates:

- The response directly addresses the query.
- Minimal unnecessary information.

3. Context Precision: measures how much of the retrieved context is actually useful for generating the answer.

Purpose:

- Identifies retrieval noise.
- Rewards retrieval systems that return highly relevant documents.

Copyright © 2026. All rights reserved

High Score Indicates:

- Most retrieved documents contribute to the final answer.
- Few irrelevant chunks are retrieved.

4. Context Recall: evaluates whether the retrieval system fetched all the information required to answer the query completely.

Purpose:

- Measures retrieval completeness.
- Ensures important facts are not missed.

High Score Indicates:

- Retrieved documents contain all necessary information.
- The generated response is complete and well-supported.

Did you know?
Many production-grade RAG applications continuously monitor metrics like **Faithfulness**, **Answer Relevance**, and **Latency** to maintain reliable performance.

Quick Fact:
Modern RAG evaluation combines **retrieval metrics**, **generation metrics**, and **LLM-based evaluation** for comprehensive assessment.

Benefits of Using RAGAS:

RAGAS provides a standardized framework for assessing retrieval and generation quality. The table below outlines its key benefits and their significance.

Copyright © 2026. All rights reserved

Table 9: Benefits of Using RAGAS

Benefit	Description
No Extensive Labeled Data Required	Evaluates RAG systems without relying on large manually labeled datasets.
Hallucination Detection	Identifies unsupported or fabricated information using faithfulness scoring.
Comprehensive Evaluation	Measures both retrieval performance and generated answer quality.
Continuous Performance Monitoring	Supports ongoing evaluation and optimization of RAG pipelines.
Standardized Benchmarking	Provides consistent metrics for comparing different RAG models and architectures.

Use Case

Title: Evaluating Retrieval-Augmented Generation (RAG) Systems Using RAGAS

Scenario:

Organizations deploy Retrieval-Augmented Generation (RAG) systems for applications such as customer support, enterprise search, healthcare assistants, and knowledge management.

Copyright © 2026. All rights reserved

While these systems improve response accuracy by retrieving external information, their performance depends on both the quality of retrieved documents and the correctness of generated answers. Without proper evaluation, RAG systems may retrieve irrelevant information, generate hallucinated responses, or provide incomplete answers.

Solution with RAGAS:

RAGAS (Retrieval-Augmented Generation Assessment) provides an automated framework for evaluating RAG systems. It measures retrieval quality and response quality using metrics such as Context Precision, Context Recall, Faithfulness, and Answer Relevance, enabling organizations to identify weaknesses and continuously improve their RAG pipelines.

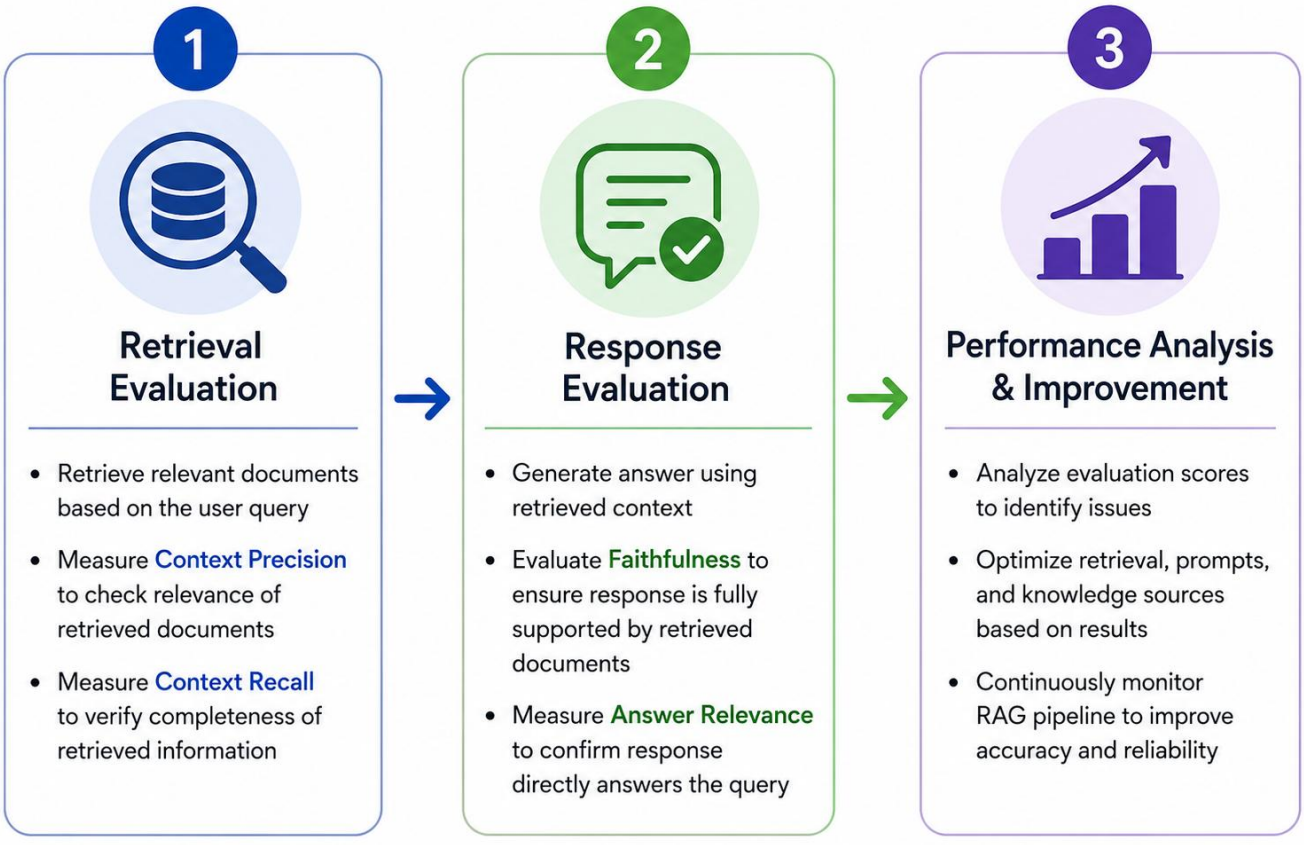


Figure. 8.25: RAGAS Evaluation Workflow

The Figure. 8.25 shows a simple three-step workflow for evaluating a RAG system using RAGAS.

Copyright © 2026. All rights reserved

1. Retrieval Evaluation

- Retrieve relevant documents or knowledge chunks based on the user's query.
- Measure Context Precision to determine whether the retrieved documents are relevant.
- Measure Context Recall to verify that all required information has been retrieved.

2. Response Evaluation

- Generate an answer using the retrieved context.
- Evaluate Faithfulness to ensure the response is fully supported by the retrieved documents.
- Measure Answer Relevance to confirm that the response directly answers the user's query.

3. Performance Analysis and Improvement

- Analyze evaluation scores to identify retrieval and generation issues.
- Optimize document retrieval, prompt design, and knowledge sources based on evaluation results.
- Continuously monitor the RAG pipeline to improve accuracy, reliability, and user satisfaction.

Copyright © 2026. All rights reserved

Benefits:

- Automatically evaluates both retrieval quality and generated responses.
- Detects hallucinations using faithfulness scoring.
- Reduces retrieval noise by measuring context precision.
- Identifies missing information through context recall.
- Supports continuous optimization and benchmarking of RAG systems.

Example:

A company deploys a RAG-based customer support assistant that answers questions using product manuals and FAQs. During evaluation, RAGAS reports low Context Precision, indicating that several irrelevant documents are being retrieved. It also detects low Faithfulness, showing that some responses include unsupported information. The engineering team improves document indexing, refines retrieval parameters, and updates prompts. After re-evaluation, the assistant retrieves more relevant documents and generates accurate, context-grounded responses, resulting in higher customer satisfaction.

Evaluating RAG systems with RAGAS enables organizations to measure retrieval effectiveness, detect hallucinations, and continuously improve the accuracy, relevance, and reliability of AI-generated responses.

8.5 Cloud-native vector search services

Cloud-native vector search services are fully managed databases or cloud services designed to store, index, and retrieve high-dimensional vector embeddings. They support semantic similarity search by comparing vectors instead of relying on exact keyword matching.

Copyright © 2026. All rights reserved

Unlike self-hosted vector databases, cloud-native services automatically handle infrastructure management, scaling, backups, security, and software updates.

Importance of Cloud-Native Vector Search Services:

Traditional databases are not optimized for searching millions of vector embeddings generated by AI models. Cloud-native vector search services solve these challenges.

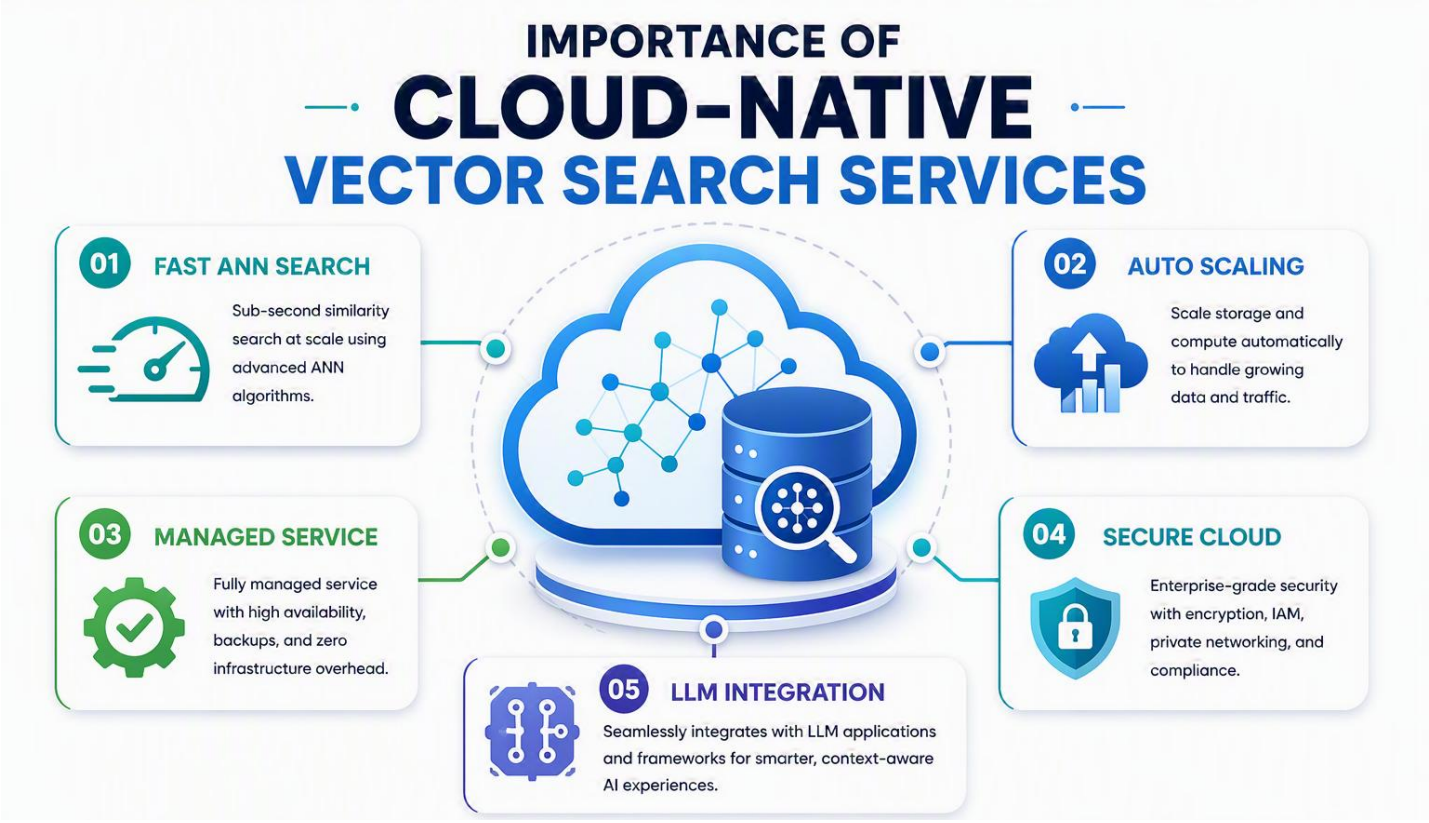


Figure. 8.26: Importance of Cloud-Native Vector Search Services

The figure highlights the core capabilities and benefits of cloud-native vector search services in modern AI and RAG systems. Cloud-native vector search services enable scalable, secure, and high-performance semantic search for modern AI applications.

- **Fast ANN Search:** Performs sub-second similarity searches using Approximate Nearest Neighbor (ANN) algorithms for rapid retrieval.

Copyright © 2026. All rights reserved

- **Automatic Scaling:** Dynamically scales compute and storage resources to handle growing data and user traffic.

- **Managed Service:** Provides built-in maintenance, backups, high availability, and monitoring with minimal operational overhead.

- **Secure Cloud Environment:** Ensures enterprise-grade security through encryption, IAM, private networking, and compliance support.

- **LLM Integration:** Seamlessly connects with Large Language Models (LLMs) to power Retrieval-Augmented Generation (RAG), semantic search, and AI assistants.

Cloud Native Architecture:

Cloud-native architecture is a modern software design approach that enables applications to be scalable, resilient, and easily deployable on cloud platforms. It combines containers, microservices, DevOps, and CI/CD to deliver reliable, high-performance applications.

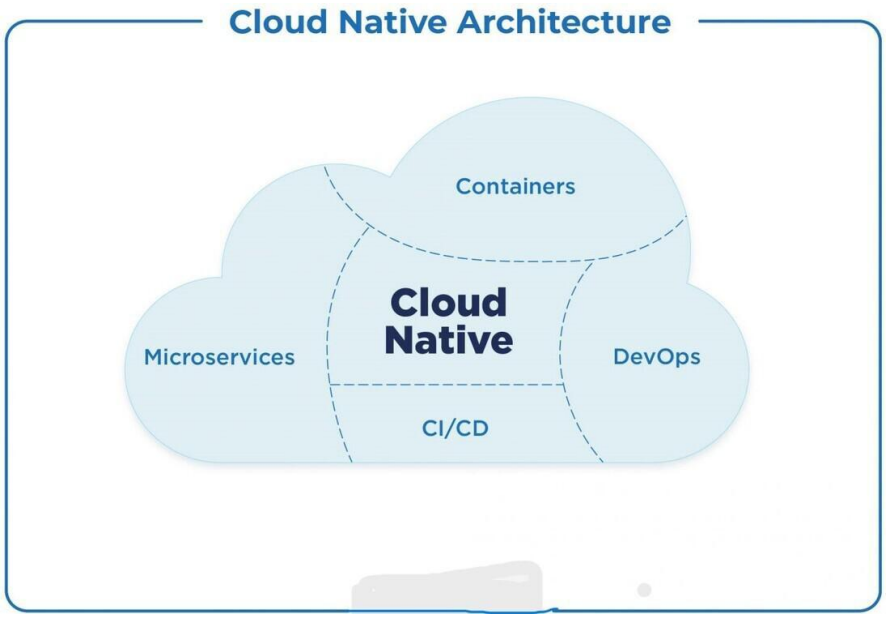


Figure. 8.27: Cloud-Native Architecture

Copyright © 2026. All rights reserved

The figure illustrates the core components of cloud-native architecture and their role in building scalable, resilient, and modern cloud applications.

- **Containers:** Containers package applications with all their dependencies, ensuring consistent execution across different environments. They enable portability, efficient resource utilization, and rapid deployment.

- **Microservices:** Applications are divided into small, independent services, each responsible for a specific function. This modular approach improves scalability, flexibility, and fault isolation.

- **DevOps:** DevOps integrates development and operations through collaboration and automation. It accelerates software delivery while improving reliability, monitoring, and operational efficiency.

- **CI/CD (Continuous Integration/Continuous Deployment):** CI/CD automates code integration, testing, and deployment, enabling faster software releases with fewer errors and consistent application updates.

Key Features of Cloud-Native Vector Search Services:

Cloud-native vector search services provide a robust and scalable platform for storing, indexing, and retrieving vector embeddings, enabling efficient semantic search in AI applications.

Copyright © 2026. All rights reserved

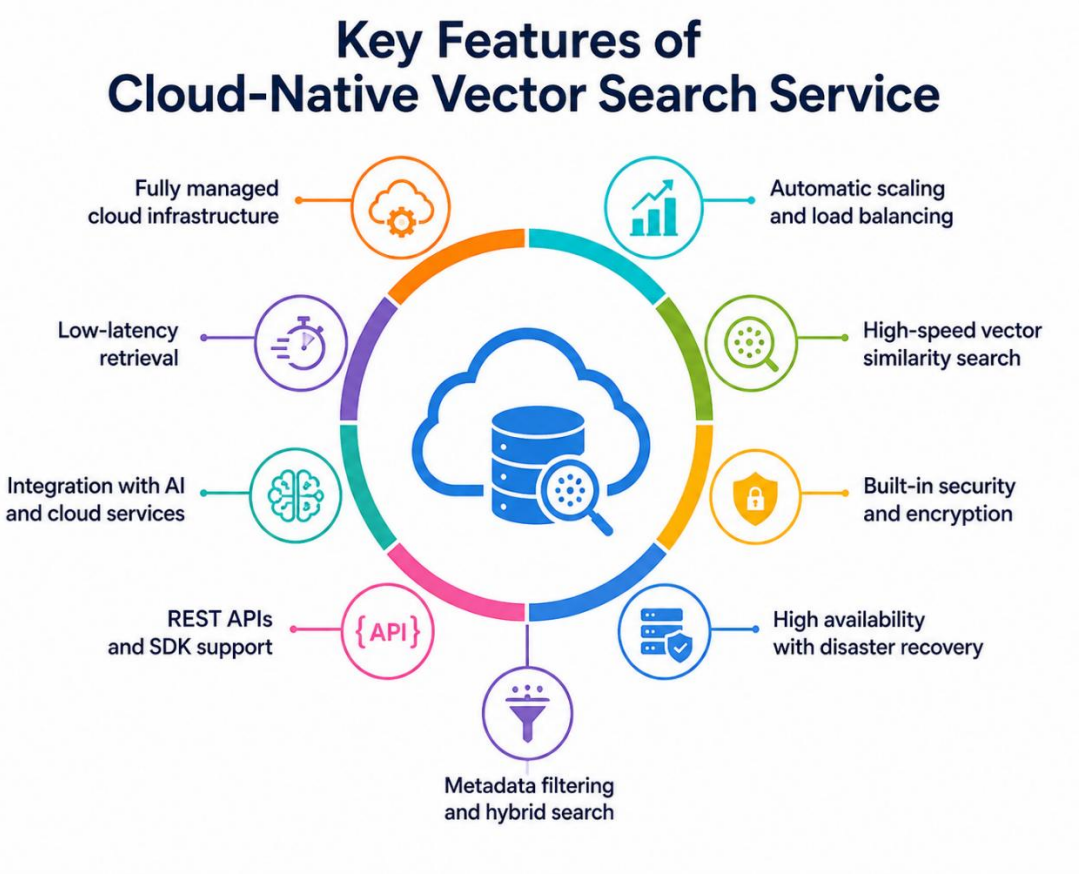


Figure. 8.28: Key Features of Cloud-Native Vector Search Services

The figure illustrates the key features of cloud-native vector search services that enable scalable, secure, and high-performance semantic search for AI applications. The table below provides an overview of the core capabilities of cloud-native vector search services that support semantic search and Retrieval-Augmented Generation (RAG) systems.

Table 10: Key Features of Cloud-Native Vector Search Services

Feature	Description
Fully Managed Cloud Infrastructure	Provides a fully managed environment with automatic maintenance, updates, backups, and minimal infrastructure management.

Copyright © 2026. All rights reserved

Automatic Scaling and Load Balancing	Dynamically scales compute and storage resources while balancing workloads to maintain consistent performance.
High-Speed Vector Similarity Search	Uses optimized indexing and Approximate Nearest Neighbor (ANN) algorithms for fast and accurate similarity search.
Built-in Security and Encryption	Ensures data protection through encryption, Identity and Access Management (IAM), network isolation, and compliance support.
High Availability with Disaster Recovery	Provides data replication, automatic failover, backup, and disaster recovery for continuous service availability.
Metadata Filtering and Hybrid Search	Combines vector similarity search with metadata filtering and keyword-based search to improve retrieval accuracy.
REST APIs and SDK Support	Offers REST APIs and SDKs for seamless integration with applications across multiple programming languages.
Integration with AI and Cloud Services	Integrates with AI frameworks, Large Language Models (LLMs), and cloud platforms to support intelligent applications.
Low-Latency Retrieval	Delivers real-time retrieval of relevant vector embeddings with minimal response time.

Copyright © 2026. All rights reserved

Monitoring and Performance Analytics	Provides monitoring, logging, and analytics to optimize system performance, resource utilization, and search efficiency.

Quick Fact:
Most cloud-native services provide built-in encryption, access control, and compliance for enterprise security.

Popular Cloud-Native Vector Search Services:

Cloud-native vector search services provide scalable, managed, and high-performance platforms for storing and retrieving vector embeddings. They are widely used in semantic search, Retrieval-Augmented Generation (RAG), recommendation systems, and other AI-powered applications. Popular cloud-native vector search services are summarized in below table.

Table 11: Popular Cloud-Native Vector Search Services

Service	Description	Key Features
Azure AI Search	Microsoft's managed search service with built-in vector search.	Hybrid search, semantic ranking, Azure OpenAI integration.
Amazon OpenSearch Service	AWS-managed search service supporting vector similarity search.	ANN search, real-time indexing, auto scaling.

Copyright © 2026. All rights reserved

Google Vertex AI Vector Search	Google Cloud's managed vector search platform for AI applications.	Low-latency search, Vertex AI integration, scalable indexing.
Pinecone	Fully managed vector database for AI and semantic search.	Fast retrieval, metadata filtering, automatic scaling.
Weaviate Cloud Service	Managed vector database with semantic and hybrid search.	Hybrid search, graph support, LLM integration.
MongoDB Atlas Vector Search	Vector search integrated into MongoDB Atlas.	Hybrid queries, automatic indexing, cloud scalability.
Elasticsearch	Distributed search engine with vector search capabilities.	Dense vectors, hybrid search, real-time analytics.

Use Case

Title: Cloud-Native Vector Search Services for Enterprise AI Applications

Scenario:

Organizations deploy AI chatbots, Retrieval-Augmented Generation (RAG) assistants, recommendation systems, and semantic search applications that require fast and accurate retrieval of relevant information from large-scale datasets. Traditional databases are not designed for similarity search on high-dimensional embeddings, leading to slow retrieval, limited scalability, and increased infrastructure management. Cloud-native vector search

Copyright © 2026. All rights reserved

services address these challenges by providing fully managed, scalable, and high-performance vector search capabilities.

Solution with AI:

Cloud-native vector search services enable organizations to store, index, and retrieve vector embeddings efficiently without managing the underlying infrastructure. These services automatically scale resources, optimize vector indexing, provide low-latency semantic search, and integrate seamlessly with AI models and cloud platforms, making them ideal for enterprise RAG and generative AI applications.

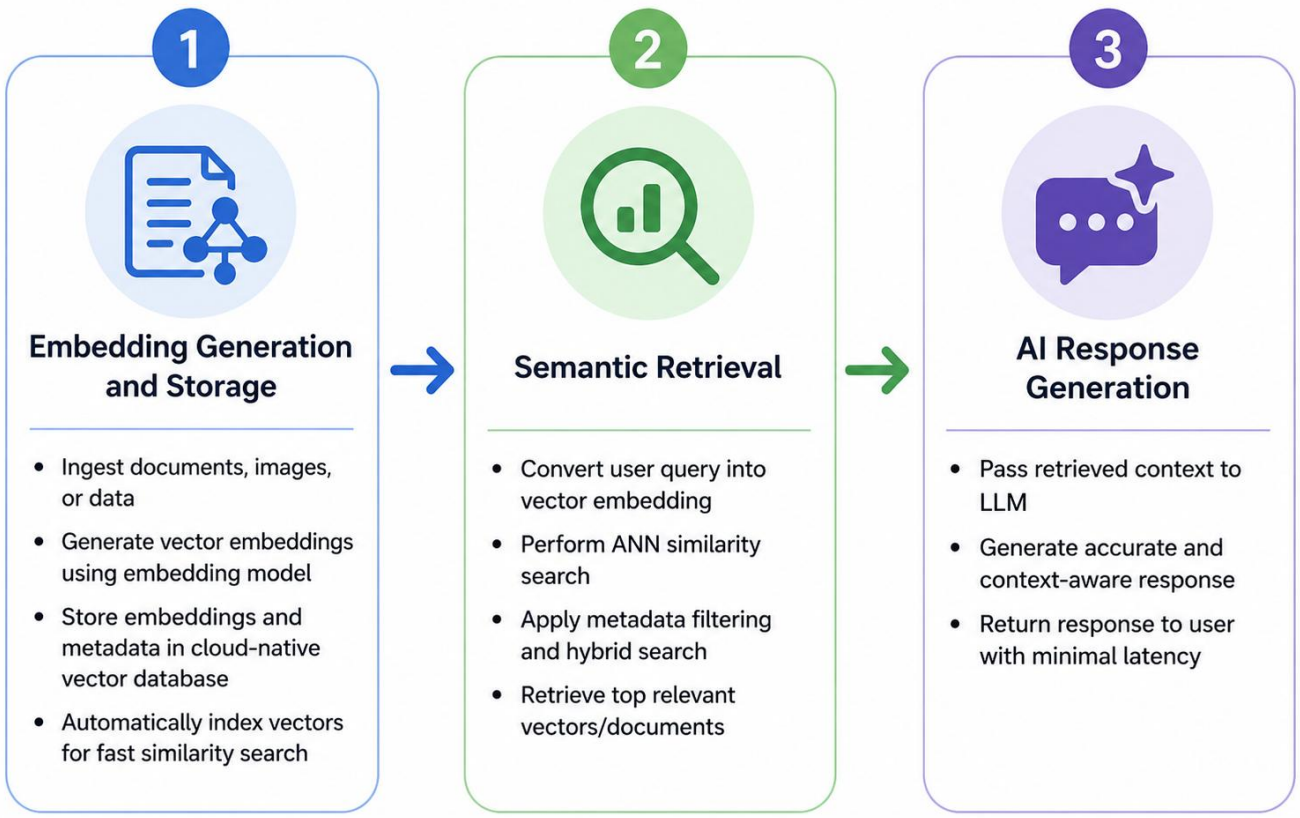


Figure. 8.29: Cloud-Native Vector Search Service Workflow

Figure. 8.29 shows a simple three-step workflow for implementing cloud-native vector search services.

Copyright © 2026. All rights reserved

1. Embedding Generation and Storage

- Convert documents, images, or other data into vector embeddings using an embedding model.
- Store embeddings and metadata in a managed cloud-native vector database.
- Automatically index vectors for efficient similarity search.

2. Semantic Retrieval

- Convert the user's query into a vector embedding.
- Perform Approximate Nearest Neighbor (ANN) search to retrieve the most relevant vectors.
- Apply metadata filtering and hybrid search to improve retrieval accuracy.

3. AI Response Generation

- Pass the retrieved information to a Large Language Model (LLM).
- Generate context-aware and accurate responses based on the retrieved content.
- Return the final response to the user with minimal latency.

Benefits:

- Fully managed infrastructure with automatic scaling.
- High-speed semantic search across millions of vectors.
- Low-latency retrieval for real-time AI applications.

Copyright © 2026. All rights reserved

- Built-in security, encryption, and high availability.
- Seamless integration with cloud services and RAG frameworks.
- Reduced operational overhead and infrastructure maintenance.

Example:

A multinational company deploys an internal AI assistant to help employees search HR policies, technical documentation, and compliance manuals. All documents are converted into vector embeddings and stored in a cloud-native vector search service such as Pinecone or Azure AI Search. When an employee asks, *"What is the company's remote work reimbursement policy?"*, the system performs semantic similarity search to retrieve the most relevant documents. The retrieved content is then passed to an LLM, which generates an accurate, context-aware response within seconds.

Cloud-native vector search services enable enterprises to build scalable, secure, and high-performance AI applications by providing efficient semantic retrieval while eliminating the complexity of managing vector database infrastructure.

8.6 Use Cases

Organizations increasingly rely on Retrieval-Augmented Generation (RAG) to unlock the value of their internal knowledge by enabling intelligent information retrieval and context-aware responses. Two of the most common applications are accurate question-answering over private documents, which allows users to obtain reliable answers from enterprise data, and enhanced search capabilities for large knowledge bases, which improve the discovery of relevant information through semantic search rather than simple keyword matching. Together, these use cases enhance productivity, reduce information retrieval time, and support informed decision-making.

Copyright © 2026. All rights reserved

1. Title: Accurate Question-Answering over Private Documents

Scenario:

A company stores thousands of confidential documents, including HR policies, technical manuals, legal contracts, and internal reports. Employees often struggle to locate the correct information using traditional keyword-based search, resulting in delays and inconsistent answers.

Solution with AI:

The organization deploys a Retrieval-Augmented Generation (RAG) assistant integrated with a cloud-native vector search service. Documents are converted into vector embeddings and securely stored in a managed vector database. When users ask questions, the system retrieves the most relevant documents using semantic search and provides accurate, context-aware answers generated by an LLM.

Copyright © 2026. All rights reserved

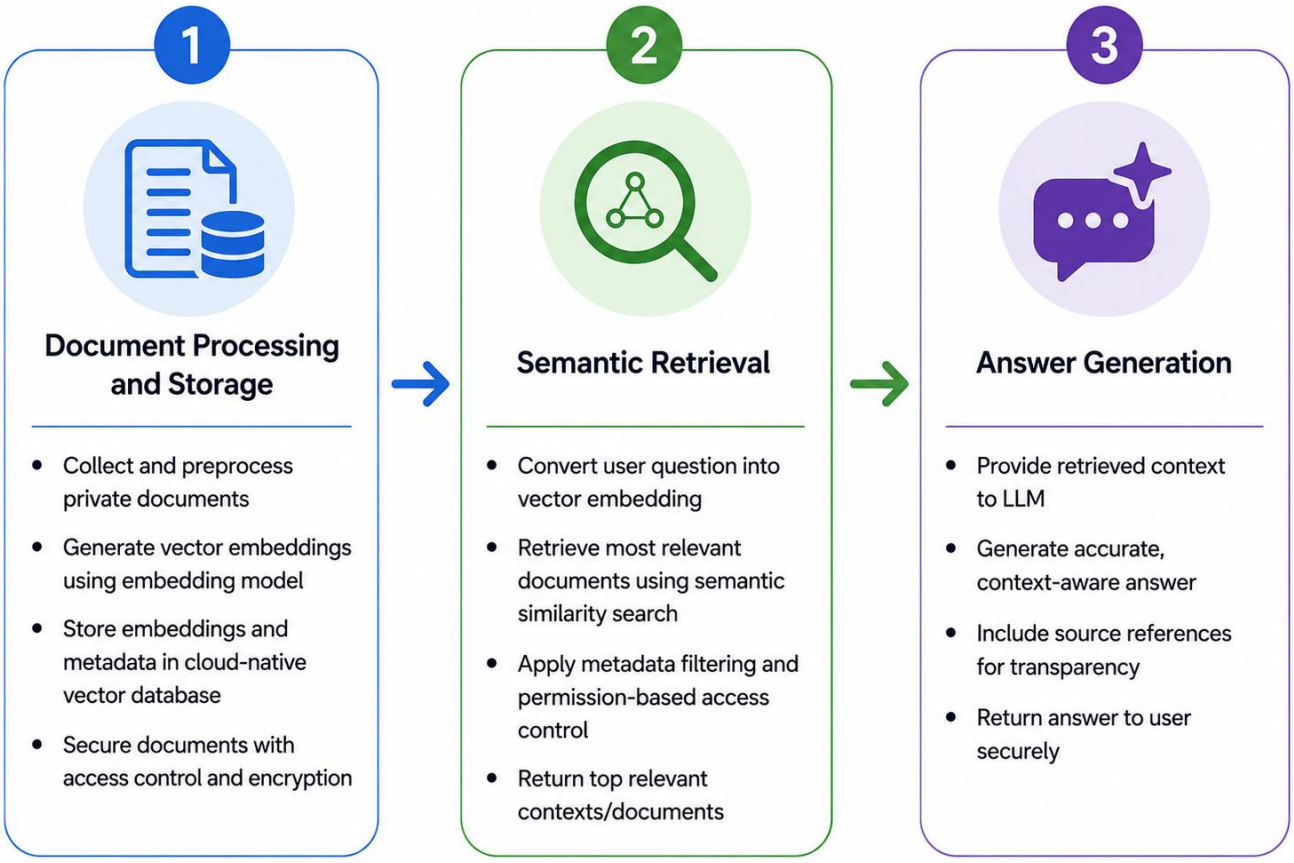


Figure. 8.30: AI-Based Question-Answering Workflow

Figure. 8.30 shows a simple three-step workflow for answering questions over private enterprise documents.

1. Document Processing and Storage

- Collect and preprocess enterprise documents.
- Generate vector embeddings using an embedding model.
- Store embeddings and metadata in a cloud-native vector database.

Copyright © 2026. All rights reserved

2. Semantic Retrieval

- Convert the user's question into a vector embedding.
- Retrieve the most relevant documents using semantic similarity search.
- Filter results based on user permissions and metadata.

3. Answer Generation

- Provide the retrieved context to the LLM.
- Generate an accurate, context-aware response.
- Return the answer with references to the source documents.

Benefits:

- Accurate answers from trusted enterprise documents.
- Secure access to private organizational knowledge.
- Reduced search time and improved employee productivity.
- Context-aware responses with minimal hallucinations.
- Scalable retrieval across millions of documents.

Copyright © 2026. All rights reserved

Example:

An employee asks, "What is the company's work-from-home reimbursement policy?" Instead of searching multiple HR documents manually, the RAG assistant retrieves the relevant policy from the enterprise knowledge base and generates a concise, accurate answer while ensuring that only authorized employees can access the information.

AI-powered question-answering over private documents enables organizations to deliver secure, accurate, and efficient access to enterprise knowledge while protecting sensitive information.

2. **Title:** Enhanced Search Capabilities for Large Knowledge Bases

Scenario:

A global enterprise maintains a massive knowledge base containing product documentation, technical manuals, customer support articles, research reports, and internal policies. Employees and support teams often struggle to find the most relevant information using traditional keyword-based search, especially when documents contain similar terminology or different phrasing.

Solution with AI:

The organization deploys a cloud-native vector search service integrated with a Retrieval-Augmented Generation (RAG) system. Documents are converted into vector embeddings and stored in a managed vector database. When users submit natural language queries, semantic similarity search retrieves the most relevant content, enabling the LLM to generate accurate and context-aware responses.

Copyright © 2026. All rights reserved

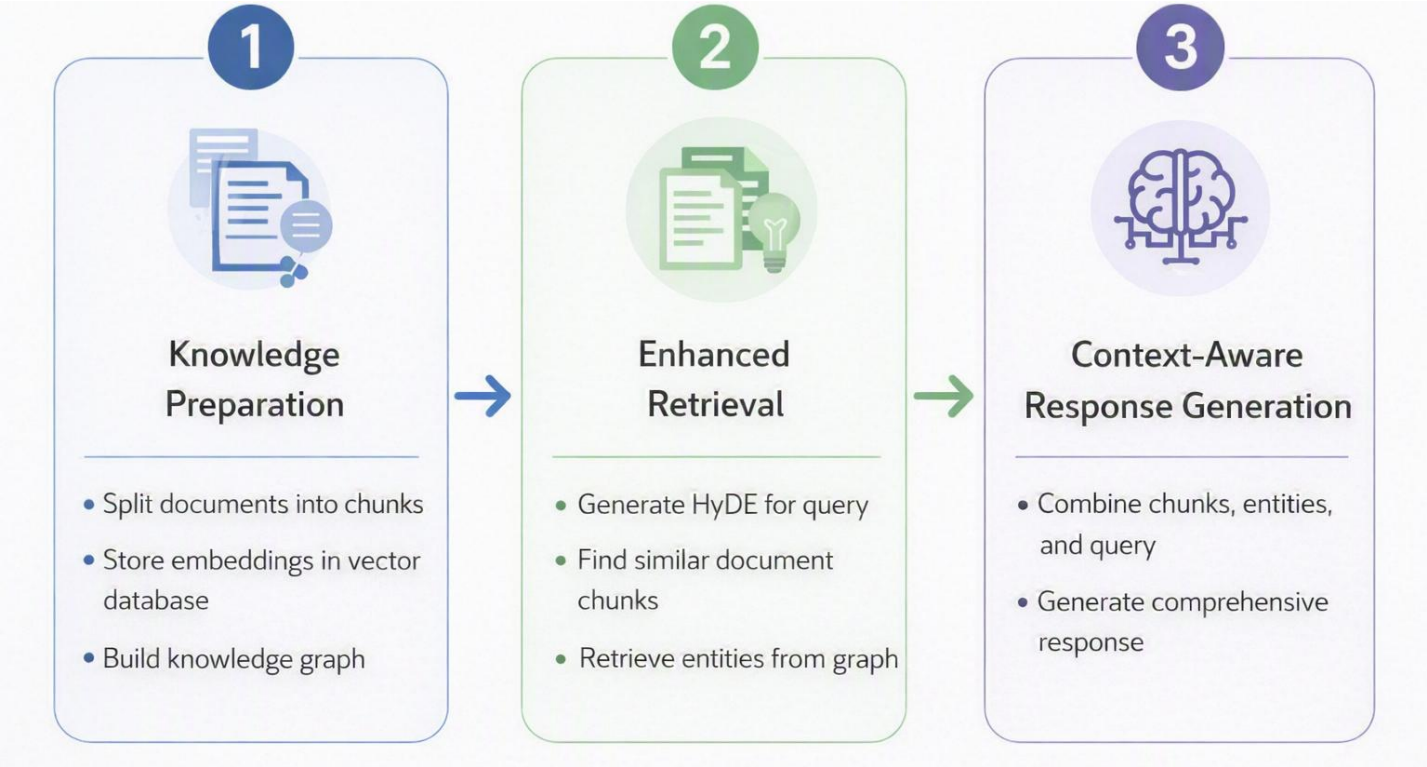


Figure. 8.31: AI-Powered Knowledge Base Search Workflow

Figure. 8.31 shows a simple three-step workflow for enhancing search across large enterprise knowledge bases.

1. Knowledge Base Indexing

- Collect and preprocess enterprise documents.
- Generate vector embeddings using an embedding model.
- Store embeddings and metadata in a cloud-native vector database.

2. Semantic Knowledge Retrieval

- Convert the user's query into a vector embedding.
- Perform semantic similarity search across the knowledge base.

Copyright © 2026. All rights reserved

- Retrieve the most relevant documents using metadata filtering.

3. Intelligent Response Delivery

- Provide the retrieved content to the LLM.
- Generate an accurate, context-aware response.
- Return the answer with links to the relevant source documents.

Benefits:

- Fast semantic search across millions of documents.
- Improved retrieval accuracy compared to keyword search.
- Reduced search time and increased productivity.
- Scalable and low-latency knowledge retrieval.
- Better user experience with context-aware responses.

Example:

A support engineer searches for "How do I configure multi-factor authentication for remote users?" Instead of matching only exact keywords, the AI system semantically retrieves the most relevant configuration guides, security policies, and troubleshooting documents from the enterprise knowledge base. The LLM then generates a concise answer with references to the original documents.

Copyright © 2026. All rights reserved

Cloud-native vector search services enable organizations to perform intelligent semantic search across large knowledge bases, improving information discovery, operational efficiency, and decision-making.

8.7 Case Studies

Case Study 1:

Retrieval-Augmented Generation (RAG) is transforming knowledge-intensive industries by combining the reasoning capabilities of generative AI models with reliable domain-specific information sources. In the legal sector, where accuracy, context, and access to authoritative documents are critical, RAG enables AI assistants to retrieve relevant legal content and generate meaningful insights. The following case study highlights how Thomson Reuters leveraged RAG-based AI capabilities to enhance legal research, improve information discovery, and support legal professionals in making faster and more informed decisions.

Title: AI-Powered Legal Research Assistant Using Retrieval-Augmented Generation (RAG)

Background:

The legal industry relies heavily on vast volumes of legal documents, including case laws, statutes, regulations, court judgments, and legal commentary. Legal professionals spend significant time searching, reviewing, and analyzing complex information to prepare arguments and provide accurate advice. Thomson Reuters developed AI-powered legal research capabilities using Retrieval-Augmented Generation (RAG) to combine large language models with trusted legal content, enabling faster and more reliable legal research while maintaining accuracy and source traceability.

Copyright © 2026. All rights reserved

- ***Organization:*** *Thomson Reuters*

- ***Target Industry:*** *Legal Services, Law Firms, Corporate Legal Departments, Compliance & Regulatory Services*

Problem Statement:

- *Legal professionals needed to analyze millions of legal documents, case precedents, and regulatory materials efficiently.*

- *Traditional keyword-based search often failed to understand legal context, intent, and relationships between cases.*

- *Generative AI systems without reliable data sources could produce inaccurate legal responses or unsupported conclusions.*

Proposed Solution:

Thomson Reuters implemented a Retrieval-Augmented Generation (RAG)-based legal AI assistant that combines enterprise legal databases with generative AI capabilities.

RAG Workflow:

1. ***Knowledge Retrieval****: Retrieves relevant legal documents, case laws, regulations, and expert commentary from Thomson Reuters' trusted legal databases.*

2. ***Context Enhancement****: Uses retrieved information as additional context for the AI model.*

3. ***AI Response Generation:*** *Generates legal summaries, research insights, and answers with references to supporting legal sources.*

Copyright © 2026. All rights reserved

4. ***Human Validation:*** *Enables lawyers to review AI-generated outputs before making legal decisions.*

Outcome:

- *Faster legal research and document analysis.*
- *Improved access to relevant case laws and legal information.*
- *Reduced time spent on manual searches and repetitive research tasks.*
- *Enhanced productivity of lawyers and legal professionals.*
- *More reliable AI responses through retrieval from authoritative legal sources.*

Key Takeaways:

- *RAG improves accuracy by grounding AI responses in verified legal information.*
- *Domain-specific knowledge bases help generative AI deliver specialized answers.*
- *Human-in-the-loop validation remains essential for high-impact professional decisions.*
- *RAG enables organizations to safely adopt generative AI while maintaining trust, transparency, and compliance.*

Reference:

- https://legal.thomsonreuters.com/en/products/cocounsel-legal
- https://www.legalcurrent.com/thomson-reuters-launches-ai-assisted-research-on-westlaw-and-additional-generative-ai-powered-solutions/

Copyright © 2026. All rights reserved

Case Study 2:

Organizations generate and manage large volumes of internal knowledge, including technical documentation, operational guidelines, employee resources, and business processes. Traditional search systems often struggle to provide accurate and context-aware answers from this growing information base. Retrieval-Augmented Generation (RAG) enables enterprises to build intelligent knowledge assistants that retrieve relevant internal information and generate precise responses using organizational data. This case study highlights how DoorDash leveraged an internal knowledge base Q&A system using RAG to improve employee access to information, reduce search time, and enhance operational efficiency.

Title: AI-Powered Internal Knowledge Base Q&A System Using Retrieval-Augmented Generation (RAG)

Background:

As a large technology-driven delivery platform, DoorDash manages extensive internal knowledge across engineering documentation, operational processes, product information, and employee resources. Employees often need quick access to accurate information across multiple repositories, but traditional search approaches can be time-consuming and may not understand the context behind user queries. By applying RAG techniques, DoorDash developed AI-powered knowledge retrieval capabilities that combine enterprise documents with generative AI to provide relevant, context-aware answers for internal users.

- ***Organization:*** *DoorDash*
- ***Target Industry:*** *Food Delivery Technology, E-commerce, Software Engineering, Enterprise Operations*

Copyright © 2026. All rights reserved

Problem Statement:

- *Employees needed faster access to internal documentation and organizational knowledge.*
- *Information was distributed across multiple sources, making manual search inefficient.*
- *Traditional keyword-based search could not understand complex questions or provide summarized answers.*
- *Employees required reliable responses based on approved internal knowledge sources.*

Proposed Solution:

DoorDash implemented an RAG-based internal knowledge assistant that connects generative AI models with company knowledge repositories.

RAG Workflow:

1. ***Knowledge Collection:*** *Collects internal documents, technical guides, FAQs, policies, and operational resources.*
2. ***Document Processing and Indexing:*** *Converts documents into searchable embeddings and stores them in a vector database.*
3. ***Contextual Retrieval:*** *Retrieves the most relevant knowledge chunks based on employee queries.*
4. ***AI-Powered Response Generation:*** *Generates concise answers using retrieved internal information.*

Copyright © 2026. All rights reserved

5. ***Continuous Improvement****: Uses user feedback to improve retrieval quality and response accuracy.*

Outcome:

- *Faster access to organizational knowledge for employees.*
- *Reduced dependency on manual searches and support teams.*
- *Improved productivity across engineering, operations, and business teams.*
- *More consistent and accurate answers from trusted internal sources.*
- *Enhanced knowledge sharing across the organization.*

Key Takeaways:

- *RAG enables enterprises to transform static documentation into interactive knowledge assistants.*
- *Combining enterprise data with generative AI improves response accuracy and relevance.*
- *Vector search and semantic retrieval help employees find information beyond keyword matching.*
- *Internal AI assistants can improve efficiency while maintaining control over organizational knowledge.*

Copyright © 2026. All rights reserved

Reference:

- https://careersatdoordash.com/blog/large-language-modules-based-dasher-support-automation/

- https://careersatdoordash.com/blog/category/engineering/data-science-and-machine-learning/gen-ai/

Copyright © 2026. All rights reserved

AICERTs®

www.aicerts.ai

Contact

252 West 37th St., Suite 1200W
New York, NY 10018